Finding similar Medium articles

Content

* Introduction to Business case : Data Scientist at Medium.

* Exploratory Data Analysis
Mean reading time vs article lengthAuthors vs article topicsASSESSMENT
[medium] Tokenization

* Data Preprocessing
Word ContractionsASSESSMENT
[easy] Word Expansions[easy] Limitations[medium] Contractions I

* How do we represent text as numbers ?
One-Hot EncodingASSESSMENT
[easy] Document Encoding

* Optimization on One Hot Encoding
Sparse vectorsBag of WordsTF - IDFASSESSMENT
[hard] Bag of Words Representation[medium] Inverse Document Frequency[medium] Count Vectorizer[hard] L1 and L2 Norm

* Euclidean distance vs Cosine Similarity
ASSESSMENT
[hard] Cosine Similarity

* How will we find similarity between documents
BOWTF-IDFASSESSMENT
[easy] TF-IDF Features

* How do we visualize the features?
T-SNEASSESSMENT
[medium] PCA Conditions

Finding similar Medium articles

* Medium is an online publishing platform which hosts a hybrid collection of blog posts from both amateur and professional people and publications.

* In 2020, about 47,000 articles were published daily on the plaform and it had about 200M visitors every month.

Problem Statement:
- You want to give readers a better reading experience at Medium. To do that, you want to that the user is reading.

* More concretely, given a Medium article find a set of similar articles.

How would a human find similar articles in a corpus ?

1. Look at the title - find similar titles.

2. Find articles by the same author.

3. Go through the text, understand it and group the aricles within broader topics.

Let's have a look at the data

What data are we going to use ?

1. Each article in Medium has a title, article text and author associated with it. To begin with, this data should be sufficient to understand the articles and to find similar articles.

2. The user might like articles which belong the same topic -

* For example: if a user is reading an article on Neural Networks, he/she might be interested in a similar article which talks about Convolutional Neural Netwroks (both of these aticles belong to a broader domain of Deep Learning).

How do we get this data ?

* Well, we can scrape it from Medium (scrapping is covered later in the track).

* Weve done that already.

* We have a collection of medium articles with its title, subtitle, author, reading time, text and the link of that article.

Code

```
# libraries to display dataframe and images
from IPython.display import display
from PIL import Image
# matplotlib for vizualization
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
# inbuild library to work with textual data
import string
# Setting up the NLTK to pre-processing textual data
import nltk
from nltk.corpus import stopwords, wordnet
from nltk.stem import PorterStemmer, LancasterStemmer, SnowballStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize, sent_tokenize
nltk.download('punkt')
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('tagsets')
nltk.download('universal_tagset')
nltk.download('treebank')
sns.set_theme(style="darkgrid")
pd.set_option("display.max_columns", 100)
%matplotlib inline
```

Error Output

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package averaged_perceptron_tagger to
[nltk_data]     /root/nltk_data...
[nltk_data]   Unzipping taggers/averaged_perceptron_tagger.zip.
[nltk_data] Downloading package tagsets to /root/nltk_data...
[nltk_data]   Unzipping help/tagsets.zip.
[nltk_data] Downloading package universal_tagset to /root/nltk_data...
[nltk_data]   Unzipping taggers/universal_tagset.zip.
[nltk_data] Downloading package treebank to /root/nltk_data...
[nltk_data]   Unzipping corpora/treebank.zip.
```

1.1 Understanding the Dataset

1. Dataset source: towardsdatascience.com

2. Total 208 articles with title, article text, author name, reading_time etc.

Link for dataset - medium_articles_v3

Code

```
!gdown 1MyOEKk_z78P8JL0mTYSerRiPLVflkVK6
```

Output

```
Downloading...
From: https://drive.google.com/uc?id=1MyOEKk_z78P8JL0mTYSerRiPLVflkVK6
To: /content/medium_articles_v3.csv
```

Using [spaCy](https://spacy.io/)

* spaCy is a free open-source library for Natural Language Processing in Python. .

Code

```
import numpy as np
import pandas as pd
import spacy
from spacy import displacy
# reading the csv data file
articles = pd.read_csv("/content/medium_articles_v3.csv")
display(articles.head(10))
print("Shape of dataframe : {}".format(articles.shape))
```

Output

```
link  \
0  https://towardsdatascience.com/ensemble-method...
1  https://towardsdatascience.com/understanding-a...
2  https://towardsdatascience.com/how-to-work-wit...
3  https://towardsdatascience.com/11-dimensionali...
4  https://towardsdatascience.com/the-time-series...
5  https://netflixtechblog.com/learning-a-persona...
6  https://towardsdatascience.com/6-data-science-...
7  https://towardsdatascience.com/transformers-ex...
8  https://medium.com/coders-camp/60-python-proje...
9  https://towardsdatascience.com/geometric-found...
title  \
0   Ensemble methods: bagging, boosting and stacking
1                     Understanding AUC - ROC Curve
2  How to work with object detection datasets in ...
3  11 Dimensionality reduction techniques you sho...
4                       The Time Series Transformer
5                   Learning a Personalized Homepage
6  6 Data Science Certificates To Level Up Your C...
7  Transformers Explained Visually (Part 2): How ...
8                60 Python Projects with Source Code
9             Geometric foundations of Deep Learning
sub_title                   author  \
0  Understanding the key concepts of ensemble lea...             Joseph Rocca
1  In Machine Learning, performance measurement i...          Sarang Narkhede
2  A comprehensive guide to defining, loading, ex...           Eric Hofesmann
3  Reduce the size of your dataset while keeping ...       Rukshan Pramoditha
4  Attention Is All You Need they said. Is it a m...      Theodoros Ntakouris
5  how to best tailor each member's homepage to m...  Netflix Technology Blog
6  Pump up your portfolio and get closer to your ...         Sara A. Metwalli
7  A Gentle Guide to the Transformer under the ho...              Ketan Doshi
8  60 Python Projects with Source code solved and...             Aman Kharwal
9  Geometric Deep Learning is an attempt to unify...        Michael Bronstein
reading_time                                       text  id
0            20  This post was co-written with Baptiste Rocca.\...   1
1             5  In Machine Learning, performance measurement i...   2
2            10  Microsoft's Common Objects in Context dataset ...   3
3            16  In both Statistics and Machine Learning, the n...   4
4             6  Attention Is All You Need they said. Is it a m...   5
5            15  by Chris Alvino and Justin Basilico\nAs we've ...   6
6             6  Because of the appeal of the field of data sci...   7
7            11  This is the second article in my series on Tra...   8
```

[Output continues below...]

```
8             2  Python has been in the top 10 popular programm...   9
9            13  This blog post was co-authored with Joan Bruna...  10
```

Output

```
Shape of dataframe : (208, 7)
```

Printing one article

* The module provides a capability to pretty-print arbitrary Python data structures in a form which can be used as input to the interpreter.

Code

```
from pprint import pprint
pprint(articles.iloc[1].to_dict(), compact=True)
```

Output

{'author': 'Sarang Narkhede',
'id': 2,
'link': 'https://towardsdatascience.com/understanding-auc-roc-curve-
68b2303cc9c5?source=tag_archive---------1----------------------',
'reading_time': 5,
'sub_title': 'In Machine Learning, performance measurement is an essential '
'task. So when it comes to a classification problem, we can '
'count on an AUC - ROC Curve. When we need to check or visualize '
'the performance...',
'text': 'In Machine Learning, performance measurement is an essential task. '
'So when it comes to a classification problem, we can count on an AUC '
'- ROC Curve. When we need to check or visualize the performance of '
'the multi-class classification problem, we use the AUC (Area Under '
'The Curve) ROC (Receiver Operating Characteristics) curve. It is one '
'of the most important evaluation metrics for checking any '
"classification model's performance. It is also written as AUROC "
'(Area Under the Receiver Operating Characteristics)\n'
'Note: For better understanding, I suggest you read my article about '
'Confusion Matrix.\n'
'This blog aims to answer the following questions:\n'
'1. What is the AUC - ROC Curve?\n'
'2. Defining terms used in AUC and ROC Curve.\n'
'3. How to speculate the performance of the model?\n'
'4. Relation between Sensitivity, Specificity, FPR, and Threshold.\n'
'5. How to use AUC - ROC curve for the multiclass model?\n'
'AUC - ROC curve is a performance measurement for the classification '
'problems at various threshold settings. ROC is a probability curve '
'and AUC represents the degree or measure of separability. It tells '
'how much the model is capable of distinguishing between classes. '
'Higher the AUC, the better the model is at predicting 0 classes as 0 '
'and 1 classes as 1. By analogy, the Higher the AUC, the better the '
'model is at distinguishing between patients with the disease and no '
'disease.\n'
'The ROC curve is plotted with TPR against the FPR where TPR is on '
'the y-axis and FPR is on the x-axis.\n'
'An excellent model has AUC near to the 1 which means it has a good '
'measure of separability. A poor model has an AUC near 0 which means '
'it has the worst measure of separability. In fact, it means it is '
'reciprocating the result. It is predicting 0s as 1s and 1s as 0s. '
'And when AUC is 0.5, it means the model has no class separation '
'capacity whatsoever.\n'
"Let's interpret the above statements.\n"
"As we know, ROC is a curve of probability. So let's plot the "
'distributions of those probabilities:\n'
'Note: Red distribution curve is of the positive class (patients with '
'disease) and the green distribution curve is of the negative '

[Output continues below...]

```
'class(patients with no disease).\n'
"This is an ideal situation. When two curves don't overlap at all "
'means model has an ideal measure of separability. It is perfectly '
'able to distinguish between positive class and negative class.\n'
'When two distributions overlap, we introduce type 1 and type 2 '
'errors. Depending upon the threshold, we can minimize or maximize '
'them. When AUC is 0.7, it means there is a 70% chance that the model '
'will be able to distinguish between positive class and negative '
'class.\n'
'This is the worst situation. When AUC is approximately 0.5, the '
'model has no discrimination capacity to distinguish between positive '
'class and negative class.\n'
'When AUC is approximately 0, the model is actually reciprocating the '
'classes. It means the model is predicting a negative class as a '
'positive class and vice versa.\n'
'Sensitivity and Specificity are inversely proportional to each '
'other. So when we increase Sensitivity, Specificity decreases, and '
'vice versa.\n'
'Sensitivity, Specificity and Sensitivity, Specificity\n'
'When we decrease the threshold, we get more positive values thus it '
'increases the sensitivity and decreasing the specificity.\n'
'Similarly, when we increase the threshold, we get more negative '
'values thus we get higher specificity and lower sensitivity.\n'
'As we know FPR is 1 - specificity. So when we increase TPR, FPR also '
'increases and vice versa.\n'
'TPR, FPR and TPR, FPR\n'
'In a multi-class model, we can plot the N number of AUC ROC Curves '
'for N number classes using the One vs ALL methodology. So for '
'example, If you have three classes named X, Y, and Z, you will have '
'one ROC for X classified against Y and Z, another ROC for Y '
'classified against X and Z, and the third one of Z classified '
'against Y and X.\n'
'Thanks for Reading.\n'
"I hope I've given you some understanding of what exactly is the AUC "
'- ROC Curve. If you like this post, a tad of extra motivation will '
'be helpful by giving this post some claps . I am always open to your '
'questions and suggestions. You can share this on Facebook, Twitter, '
'Linkedin, so someone in need might stumble upon this.\n'
'You can reach me at:\n'
'LinkedIn: https://www.linkedin.com/in/narkhedesarang/\n'
'Twitter: https://twitter.com/narkhede_sarang\n'
'Github: https://github.com/TheSarang\n',
'title': 'Understanding AUC - ROC Curve'}
```

Looking at the statistical summary of the dataframe.

* This includes count, mean, median (or 50th percentile) standard variation, min-max, and percentile values of columns.

Code

```
articles.describe(include='all')
```

Output

```
link  \
count                                                  208
unique                                                 208
top     https://towardsdatascience.com/ensemble-method...
freq                                                     1
mean                                                   NaN
std                                                    NaN
min                                                    NaN
25%                                                    NaN
50%                                                    NaN
75%                                                    NaN
max                                                    NaN
title  \
count                                                  208
unique                                                 208
top     Ensemble methods: bagging, boosting and stacking
freq                                                     1
mean                                                   NaN
std                                                    NaN
min                                                    NaN
25%                                                    NaN
50%                                                    NaN
75%                                                    NaN
max                                                    NaN
sub_title        author  \
count                                                  208           208
unique                                                 204           179
top     Update: This article is part of a series. Chec...  Adam Geitgey
freq                                                     4             5
mean                                                   NaN           NaN
std                                                    NaN           NaN
min                                                    NaN           NaN
25%                                                    NaN           NaN
50%                                                    NaN           NaN
75%                                                    NaN           NaN
max                                                    NaN           NaN
reading_time                                                text  \
count     208.000000                                              208
unique           NaN                                              208
top              NaN  This post was co-written with Baptiste Rocca.\...
freq             NaN                                                1
mean       12.375000                                              NaN
```

[Output continues below...]

```
std        13.880224                                                  NaN
min         2.000000                                                  NaN
25%         6.000000                                                  NaN
50%         9.000000                                                  NaN
75%        13.000000                                                  NaN
max       149.000000                                                  NaN
id
count   208.000000
unique         NaN
top            NaN
freq           NaN
mean    107.091346
std      62.575453
min       1.000000
25%      52.750000
50%     107.500000
75%     162.250000
max     214.000000
```

* is the only integer variable.

* Lets quickly look at the ditsribution of reading times in our corpus.

Code

```
# ditsribution of reading times in our corpus
fig, axes = plt.subplots(figsize = (8, 6))
# creating histograms
sns.histplot(articles["reading_time"], kde=True, ax = axes)
# Computing percentile of the reading_time data.
first_q = np.percentile(articles["reading_time"], 25)
# Computing median (50th percentile) of the reading_time data.
second_q = np.percentile(articles["reading_time"], 50)
third_q = np.percentile(articles["reading_time"], 75)
# green lines for 25th and 75th percentile
plt.axvline(first_q, color = "green")
# red line for median reading_time
plt.axvline(second_q, color = "red")
plt.axvline(third_q, color = "green")
# plot title
plt.suptitle("Distribution of reading time across articles")
plt.show()
```

Output Image

* The graph is clearly right skewed.

* Hence most of ariticles in our corpus have a less reading time, with some articles having reading in hours.

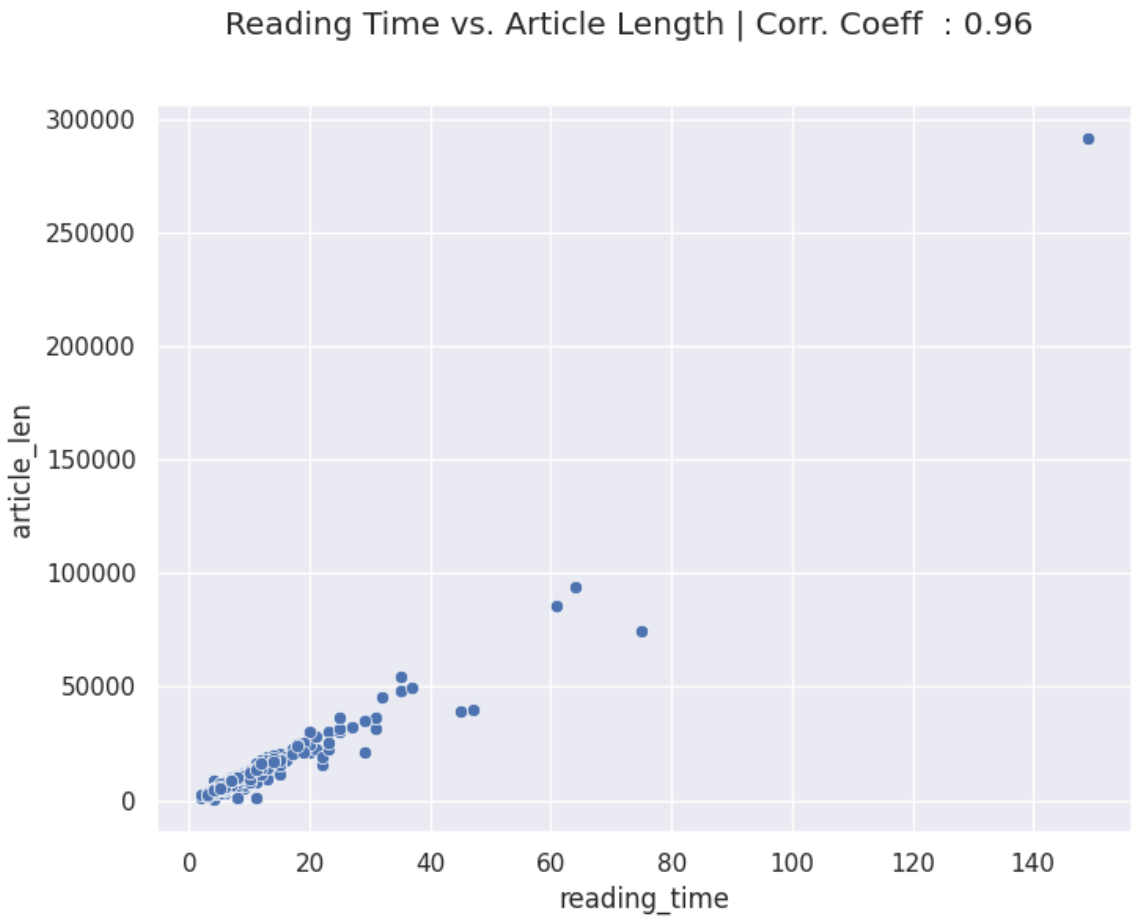
* The median reading time is 9 minutes.

How does the reading time relates with article length ?

Code

```
from scipy.stats import pearsonr
articles["article_len"] = articles["text"].apply(lambda x : len(x))
# creating scatterplot
fig, axes = plt.subplots(figsize = (8, 6))
sns.scatterplot(x = articles["reading_time"], y = articles["article_len"])
# Pearson correlation coefficient measures the linear relationship between two set of values.
corr_coeff, _ = pearsonr(articles["reading_time"], articles["article_len"])
# plot title
plt.suptitle("Reading Time vs. Article Length | Corr. Coeff  : {}".format(round(corr_coeff,
2)))
```

Output Image



* As expected, the reading time and article length are correlated.

****Do writers usually write articles on similar topics ?****

Lets have a look at the authors now. Authors usually have a preference to write on the same topic.

Code

```
for author, count in dict(articles["author"].value_counts()).items():
if(count < 2):
continue
print("Articles by {} :".format(author))
for title in articles[articles["author"] == author]["title"].values:
print(title)
print("-"*120)
```

Output

```
Articles by Adam Geitgey :
Machine Learning is Fun Part 5: Language Translation with Deep Learning and the Magic of
Sequences
Machine Learning is Fun! Part 4: Modern Face Recognition with Deep Learning
Machine Learning is Fun! Part 3: Deep Learning and Convolutional Neural Networks
Machine Learning is Fun! Part 2
Machine Learning is Fun Part 6: How to do Speech Recognition with Deep Learning
--------------------------------------------------------------------------------------------
--------------------------
Articles by Joseph Rocca :
Ensemble methods: bagging, boosting and stacking
Understanding Variational Autoencoders (VAEs)
Understanding Generative Adversarial Networks (GANs)
--------------------------------------------------------------------------------------------
--------------------------
Articles by Natassha Selvaraj :
I tripled my income with data science. Here's how.
How to Land a Data Analytics Job in 6 Months
Top 10 Data Science Projects for Beginners
--------------------------------------------------------------------------------------------
--------------------------
Articles by Mahmoud Harmouch :
17 Clustering Algorithms Used In Data Science and Mining
17 types of similarity and dissimilarity measures used in data science.
--------------------------------------------------------------------------------------------
--------------------------
Articles by Ketan Doshi :
Transformers Explained Visually (Part 2): How it works, step-by-step
Transformers Explained Visually (Part 3): Multi-head Attention, deep dive
--------------------------------------------------------------------------------------------
--------------------------
Articles by Aman Kharwal :
60 Python Projects with Source Code
180 Data Science and Machine Learning Projects with Python
--------------------------------------------------------------------------------------------
--------------------------
Articles by Sara A. Metwalli :
6 Data Science Certificates To Level Up Your Career
6 Machine Learning Certificates to Pursue in 2021
--------------------------------------------------------------------------------------------
--------------------------
Articles by Dario Radecic :
Are The New M1 Macbooks Any Good for Data Science? Let's Find Out
Top 3 Reasons Why I Sold My M1 Macbook Pro as a Data Scientist
--------------------------------------------------------------------------------------------
--------------------------
Articles by Chris Zaire :
```

[Output continues below...]

```
Articles by Rukshan Pramoditha :
11 Dimensionality reduction techniques you should know in 2021
20 Necessary Requirements of a Perfect Laptop for Data Science and Machine Learning Tasks
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Madison Hunter :
15 Habits I Learned from Highly Effective Data Scientists
How to Study for the Google Data Analytics Professional Certificate
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Terence Shin :
OVER 100 Data Scientist Interview Questions and Answers!
50+ Statistics Interview Questions and Answers for Data Scientists for 2022
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Will Koehrsen :
Hyperparameter Tuning the Random Forest in Python
Random Forest in Python
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Blake Ross :
Wealthfront:Silicon Valley Tech at Wall Street Prices
Heroes Give, Superheroes Borrow
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Alexandr Nellson :
How to invest in Bitcoin properly. Blockchain and other cryptocurrencies
How to store Bitcoins and other cryptocurrencies properly.
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Yueh :
Richart + @GoGo
2018  & : KOKO COMBO icash
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Mercatus Center :
Nassim Nicholas Taleb on Self-Education and Doing the Math (Ep. 41  Live at Mercatus)
Cliff Asness on Marvel vs. DC and Why Never to Share a Gym with Cirque du Soleil (Ep. 5  Live
at Mason)
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Susan Li :
Building A Logistic Regression in Python, Step by Step
An End-to-End Project on Time Series Analysis and Forecasting with Python
--------------------------------------------------------------------------------------------------
-------------------------
Articles by Andrei Tapalaga  :
```

[Output continues below...]

```
How to Crush the Crypto Market, Quit Your Job, Move to Paradise and Do Whatever You Want the
Rest of Your Life
Why People Still Don't Get Cryptocurrency
--------------------------------------------------------------------------------------------
-------------------------
Articles by Serge Faguet :
How to biohack your intelligence  with everything from sex to modafinil to MDMA
I'm 32 and spent $200k on biohacking. Became calmer, thinner, extroverted, healthier &
happier.
--------------------------------------------------------------------------------------------
-------------------------
```

* In the data that we have, it does seem that authors tend to write multiple articles on the same topic.

* But also, there are multiple authors writing on the same topic,

* For example:
> 1. Adam Geitgey & Susan Li, both seems to write on Machine Learning.
> 2. Natassha Selvaraj, Sara A. Metwalli & Terence Shin, all seems to write on buidling career as a Data Scientist.

##Solving common preprocessing Problems in NLP.

Expanding Word Contractions

* Contractions are the shortened word of english words or phrases.
> Example -
>
>
>

* Why is this a problem?
> Because of the following reasons -
> 1. They add in a special apostrophe character (computers cannot understand its meaning) to the text.
> 2. They are a combination of two words, hence can cause problems in tokenization.

* How to deal with it ?
> 1. Create a custom mapping of expansions.
> 2. Using contractions library.

Code

```
import re
# creating custom mapping
custom_mapping = {
"n't" : " not",
"'re" : " are",
"'ve" : " have",
"'ll" : " will",
"'m" : " am"
}
# sample data
sample_text = """
I've decided to go to the party after all. I'll reach by 05:00 PM.
He's not coming with us.
It's his birthday and he has other plans.
They've thought about going to the movies.
I won't be going to movies.
"""
expanded_text = sample_text
for x in custom_mapping.keys():
expanded_text = re.sub(x, custom_mapping[x], expanded_text)
print(expanded_text)
```

Output

```
I have decided to go to the party after all. I will reach by 05:00 PM.
He's not coming with us.
It's his birthday and he has other plans.
They have thought about going to the movies.
I wo not be going to movies.
```

* The problem with above approach

> *The number of contractions can be large to list all.There can be some not so simple cases which breaks the rule, like wont.*

> *Lets try out second approach.*

Using contractions library:
Contractions Github

Code

```
!pip install contractions
```

Output

```
Collecting contractions
Downloading contractions-0.1.73-py2.py3-none-any.whl.metadata (1.2 kB)
Collecting textsearch>=0.0.21 (from contractions)
Downloading textsearch-0.0.24-py2.py3-none-any.whl.metadata (1.2 kB)
Collecting anyascii (from textsearch>=0.0.21->contractions)
Downloading anyascii-0.3.3-py3-none-any.whl.metadata (1.6 kB)
Collecting pyahocorasick (from textsearch>=0.0.21->contractions)
Downloading pyahocorasick-2.2.0-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (13 kB)
Downloading contractions-0.1.73-py2.py3-none-any.whl (8.7 kB)
Downloading textsearch-0.0.24-py2.py3-none-any.whl (7.6 kB)
Downloading anyascii-0.3.3-py3-none-any.whl (345 kB)
???????????????????????????????????????? 345.1/345.1 kB 7.0 MB/s eta 0:00:00
 [?25hDownloading pyahocorasick-2.2.0-cp312-cp312-
manylinux_2_17_x86_64.manylinux2014_x86_64.whl (114 kB)
???????????????????????????????????????? 114.9/114.9 kB 8.9 MB/s eta 0:00:00
```

Code

```
import contractions
expanded_text = contractions.fix(sample_text)
print(expanded_text)
```

Output

```
I have decided to go to the party after all. I will reach by 05:00 PM.
He is not coming with us.
It is his birthday and he has other plans.
They have thought about going to the movies.
I will not be going to movies.
```

Observe how efficient this library is.

How do we represent text as numbers ?

* Machine learning algorithms need input data as numbers.

* What we have here is textual data.

* So we need to find a way to convert these texts into numbers (or vectors).

* Imagine each document that we have, is represented as a vector of numbers, of size N.

* Now once we have vectorized, each document we can use them as features for any machine learning model of our choice.

* There are multiple ways using which we can get a vector representation of text. Lets have a look at a couple of them.

What could be the simplest techniques of converting a document into a vector?

One-Hot encoding:

* Assigns 0 to all elements in a vector except for one, which has a value of 1. .

Step 1 - Create a vocabulary from the above corpus

> *Vocabulary is simply a set of all unique words in the documents. For the above example, the vocabulary would be*

* Lecture
* on

* *text*
* *representation*

Step 2 - Each word in the sentence would be represented as below:

Step 3 - The entire sentence is then represented as:

* Intuition behind it is that each bit represents a possible category.

* And if a particular variable cannot fall into multiple categories, then a single bit is enough to represent it.

Code

```
from sklearn.preprocessing import OneHotEncoder
import itertools
# two example documents
docs = ["cat","dog","bat","ate","lion","dog","bat"]
# split documents to tokens
tokens_docs = [doc.split(" ") for doc in docs]
# convert list of of token-lists to one flat list of tokens
# and then create a dictionary that maps word to id of word,
all_tokens = itertools.chain.from_iterable(tokens_docs)
word_to_id = {token: idx for idx, token in enumerate(set(all_tokens))}
# convert token lists to token-id lists
token_ids = [[word_to_id[token] for token in tokens_doc] for tokens_doc in tokens_docs]
# convert list of token-id lists to one-hot representation
vec = OneHotEncoder(categories="auto")
X = vec.fit_transform(token_ids)
print(X.toarray())
```

Output

```
[[0. 0. 0. 1. 0.]
[1. 0. 0. 0. 0.]
[0. 1. 0. 0. 0.]
[0. 0. 1. 0. 0.]
[0. 0. 0. 0. 1.]
[1. 0. 0. 0. 0.]
[0. 1. 0. 0. 0.]]
```

Lets optimize the OHE representation

Optimizing One Hot Encoding using Sparse vectors

* Only storing indicies of each word, to save space

Code

```
from sklearn.preprocessing import OneHotEncoder
import itertools
# two example documents
docs = ["cat","dog","bat","ate","lion","dog","bat"]
# split documents to tokens
tokens_docs = [doc.split(" ") for doc in docs]
# convert list of of token-lists to one flat list of tokens
# and then create a dictionary that maps word to id of word,
all_tokens = itertools.chain.from_iterable(tokens_docs)
word_to_id = {token: idx for idx, token in enumerate(set(all_tokens))}
# convert token lists to token-id lists
token_ids = [[word_to_id[token] for token in tokens_doc] for tokens_doc in tokens_docs]
token_ids
```

Output

```
[[3], [0], [1], [2], [4], [0], [1]]
```

How can we further optimize on One Hot Encoding ?

Can we use the frequency of each word in the document ?

* Bag Of Words: Puts .

How do we do it?

Step 1 - Create a vocabulary from the above corpus

Vocabulary is simply a set of all unique words in the documents. For the above example, the vocabulary would be -
* it
* was
* the
* best
* of
* times
* worst
* age
* wisdom
* and
* foolishness

Step 2 - Construct One Hot Encoded representation of the words

For every sentence in a text, create a OHE representation of it using the vocabulary from Step 1.

For the 1st sentence -

Similarly, for the 2nd sentence -

Step 3 - Combine the OHE word representations for every document

Once we have the OHE of all words in a document, the only thing left is to combine them, to get a get a single vector representation for the document. The are two common ways to do so -
1. Using a binary OR operator between the OHE vectors. The final document vector that we get in this case simply tells the absence or presence of certain words in the document.

2. Using a vector sum opeartor. The final document vector that we get in this case tells the frequency of each word in the document.

Lets apply both the combination techniques in the 3rd sentence -

Note: The combination using the sum operator is more commonly used, as it conveys more information about the text - like which words are used more frequently (this is obviously done after removing stopwords).

The representation of all the 3 texts above are -

* Here, we see that each of 3 documents are represented as a vector of size 11 (vocabulary size).

* This vectorization technique is easy to implement as well. Plus it also has a implementation in the scikit-learn package.

The above representation is called Bag-of-Words (BOW)

BOW is one of the simplest techniques use for textual feature extraction. Here, yet again we break the document into its smallest component - words, and build it from there.

* It does not take into account the word order or lexical information for text representation.

* The intuition is that documents having similar words are similar irrespective of the word positioning.

Code

```
import nltk
nltk.download('punkt_tab')
nltk.download('punkt')
# Custom implementation of BoW
# sample data
corpus = [
"it was the best of times",
"it was the worst of times",
"it was the age of wisdom and the age of foolishness"
]
def get_bow_representation(corpus, frequency = True):
vocabulary = set([x for x in " ".join(corpus).lower().split(" ")])
bow_rep = []
for sentence in corpus:
sentence_rep = dict([(v,0) for v in vocabulary])
for word in word_tokenize(sentence.lower()):
if frequency:
sentence_rep[word] += 1
else:
sentence_rep[word] = 1
bow_rep.append(sentence_rep)
return bow_rep
bow_representation = get_bow_representation(corpus, True)
df = pd.DataFrame(bow_representation)
df.index = corpus
display(df)
```

Error Output

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt_tab.zip.
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
```

Output

```
wisdom  best  of  age  \
it was the best of times                                 0     1   1    0
it was the worst of times                                0     0   1    0
it was the age of wisdom and the age of foolish...       1     0   2    2
worst  times  was  and  \
it was the best of times                                0      1    1    0
it was the worst of times                               1      1    1    0
it was the age of wisdom and the age of foolish...      0      0    1    1
it  foolishness  the
it was the best of times                              1            0    1
it was the worst of times                             1            0    1
it was the age of wisdom and the age of foolish...    1            1    2
```

Using [Sklearn CountVectorizer](https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html)

* It the corpus of multiple sentences (say product reviews) into a & fills it with the frequency of each word in a sentence.

Code

```
# Using CountVectorizer from scikit-learn
from sklearn.feature_extraction.text import CountVectorizer
cv = CountVectorizer()
# Learn the vocabulary dictionary and return document-term matrix
bow_rep = cv.fit_transform(corpus).todense()  # todense() returns a matrix
# create dataframe
df = pd.DataFrame(bow_rep, columns=cv.get_feature_names_out(), index=corpus)
# Get output feature names for dataframe columns.
display(df)
```

Output

```
age  and  best  \
it was the best of times                             0    0     1
it was the worst of times                            0    0     0
it was the age of wisdom and the age of foolish...   2    1     0
foolishness  it  of  the  \
it was the best of times                                 0   1   1    1
it was the worst of times                                0   1   1    1
it was the age of wisdom and the age of foolish...       1   1   2    2
times  was  wisdom  worst
it was the best of times                               1    1       0      0
it was the worst of times                              1    1       0      1
it was the age of wisdom and the age of foolish...     0    1       1      0
```

* CountVectorizer has a lot of inbuilt text processing features.

* One of them is removing stop words directly from the corpus.

Code

```
# using CountVectorizer for removing stop-words directly from the corpus.
cv = CountVectorizer(stop_words="english")
bow_rep = cv.fit_transform(corpus).todense()
df = pd.DataFrame(bow_rep)
df.columns = cv.get_feature_names_out()
df.index = corpus
display(df)
```

Output

```
age  best  foolishness  \
it was the best of times                              0     1           0
it was the worst of times                             0     0           0
it was the age of wisdom and the age of foolish...    2     0           1
times  wisdom  worst
it was the best of times                                1       0      0
it was the worst of times                               1       0      1
it was the age of wisdom and the age of foolish...      0       1      0
```

Problems:

1. The BOW representation considers each word to be equally important (in the final document vector, each word in the text has an equal contribution of 1 at one of the positions).

2. It can not distinguish between the rare important words and the common words.

* For example:
> If youre looking at a set of articles about deep learning, then the phrase neural network might be present in a lot of articles and hence does not convey a lot of information (considering the corpus).

But the phrase Reinforcement Learning might be present in a few, and hence does convey unique information about those specific articles.

So, is there a way we can create word vectors with rare and important words getting more weightage ?

Advanced BOW

* To suppress the very high-frequency words & ignore the low-frequency words, there is a need to normalize the weights of the words accordingly.

[What is TF-IDF?](https://towardsdatascience.com/tf-idf-for-document-ranking-from-scratch-in-python-on-real-world-dataset-796d339a4089)

TF-IDF is another very popular technique to get the feature representation for a text in a corpus. Similar to the BOW model, here as well the size of final document vector is equal to the number of tokens considered.

As the name suggests, TF-IDF is mainly composed of two components - TF (Term Frequency) and IDF (Inverse Document Frequency).

What is Term Frequency (TF) in TF-IDF?

* Term Frequency is the measure of how common a word (or token) is in the document.

* More (or tokens) would have a .This is calculated for every word in a document.

* There are various ways to determine the term frequency. One of the most common formulation of TF is -

Here,
* $TF(t,d)$ is the term Frequency of term t in document d
* $f_{t,d}$ is the frequency of term t in document d

The above formulation is simply the ratio of the frequency of a term in the document to the total number of terms in the document. TF of the term increases as the frequency of the term increases.

Some other formulations for Term Frequency are -
1. Using the raw count itself, $TF(t,d) = f_{t,d}$
2. Using boolean frequency, $TF(t,d) = 1$ if t occurs in d else 0
3. Logarithmically scaled frquencies, $TF(t,d) = log(1 + f_{t,d})$
4. Augmented frequency to prevent bias towards longer documents,

Using the same example from before, lets computer the TF for each word in the corpus -
1. It was the best of times
2. It was the worst of times
3. It was the age of wisdom and the age of foolishness

Code

```
# sample data
corpus = [
"it was the best of times",
"it was the worst of times",
"it was the age of wisdom and the age of foolishness"
]
def get_term_frequency(corpus):
vocabulary = set([x for x in " ".join(corpus).lower().split(" ")])
term_freq = []
for sentence in corpus:
sentence_tf = dict([(v,0) for v in vocabulary])
for word in word_tokenize(sentence.lower()):
sentence_tf[word] += 1
for v in vocabulary:
sentence_tf[v] /= len(word_tokenize(sentence))
term_freq.append(sentence_tf)
return term_freq
term_freq = get_term_frequency(corpus)
df = pd.DataFrame(term_freq)
df.index = corpus
display(df)
```

Output

```
wisdom      best  \
it was the best of times                            0.000000  0.166667
it was the worst of times                           0.000000  0.000000
it was the age of wisdom and the age of foolish...  0.090909  0.000000
of       age  \
it was the best of times                            0.166667  0.000000
it was the worst of times                           0.166667  0.000000
it was the age of wisdom and the age of foolish...  0.181818  0.181818
worst     times  \
it was the best of times                            0.000000  0.166667
it was the worst of times                           0.166667  0.166667
it was the age of wisdom and the age of foolish...  0.000000  0.000000
was       and  \
it was the best of times                            0.166667  0.000000
it was the worst of times                           0.166667  0.000000
it was the age of wisdom and the age of foolish...  0.090909  0.090909
it  foolishness  \
it was the best of times                            0.166667     0.000000
it was the worst of times                           0.166667     0.000000
it was the age of wisdom and the age of foolish...  0.090909     0.090909
the
it was the best of times                            0.166667
it was the worst of times                           0.166667
it was the age of wisdom and the age of foolish...  0.181818
```

What is Inverse Document Frequency (IDF) in TF-IDF?

* Inverse Document Frequency measures .

* A (or token) would have a . There are multiple ways to determine IDF as well.

* One of the most common formulation is -

Here,
* $IDF(t,d,C)$ is the Inverse Document Frequency of term t in corpus C. As we can see from the formulation, IDF is calculated for each word at corpus level and not for individual documents.
* $|d|$ is the absolute number of documents in the corpus.
* $|{d \in C : t \in d}|$ is the number of documents in corpus C which contains the term t

The above formulation is simply the log scaled inverse fraction of documents which contains the term t in the corpus D.

Log is used as it dampens the effect of huge number of documents in the corpus.

Some other formulations of Inverse Document Frequency are -
1. Smoothened IDF,
2. Max IDF,

Code

```
def get_inverse_document_frequency(corpus):
vocabulary = set([x for x in " ".join(corpus).lower().split(" ")])
n = len(corpus)
inverse_document_frequency = {}
for v in vocabulary:
num_docs = 0
for sentence in corpus:
if v in word_tokenize(sentence.lower()):
num_docs += 1
inverse_document_frequency[v] = np.log(n/num_docs)
return inverse_document_frequency
inverse_document_frequency = get_inverse_document_frequency(corpus)
inverse_document_frequency
```

Output

```
{'wisdom': np.float64(1.0986122886681098),
'best': np.float64(1.0986122886681098),
'of': np.float64(0.0),
'age': np.float64(1.0986122886681098),
'worst': np.float64(1.0986122886681098),
'times': np.float64(0.4054651081081644),
'was': np.float64(0.0),
'and': np.float64(1.0986122886681098),
'it': np.float64(0.0),
'foolishness': np.float64(1.0986122886681098),
'the': np.float64(0.0)}
```

Code

```
def get_tf_idf(corpus):
tf = get_term_frequency(corpus)
idf = get_inverse_document_frequency(corpus)
tf_idf = []
for tf_dict in tf:
tf_idf_sentence = {}
for t, term_freq in tf_dict.items():
tf_idf_sentence[t] = term_freq * idf[t]
tf_idf.append(tf_idf_sentence)
return tf_idf
tf_idf = get_tf_idf(corpus)
df = pd.DataFrame(tf_idf)
df.index = corpus
display(df)
```

Output

```
wisdom      best   of  \
it was the best of times                              0.000000  0.183102  0.0
it was the worst of times                             0.000000  0.000000  0.0
it was the age of wisdom and the age of foolish...    0.099874  0.000000  0.0
age     worst  \
it was the best of times                              0.000000  0.000000
it was the worst of times                             0.000000  0.183102
it was the age of wisdom and the age of foolish...    0.199748  0.000000
times  was       and  \
it was the best of times                              0.067578  0.0  0.000000
it was the worst of times                             0.067578  0.0  0.000000
it was the age of wisdom and the age of foolish...    0.000000  0.0  0.099874
it  foolishness  the
it was the best of times                              0.0     0.000000  0.0
it was the worst of times                             0.0     0.000000  0.0
it was the age of wisdom and the age of foolish...    0.0     0.099874  0.0
```

Code

```
from sklearn.feature_extraction.text import TfidfVectorizer
# using inbuilt TfidfVectorizer() function to calculate TF-IDF
tf_idf_vectorizer = TfidfVectorizer()
tf_idf_rep = tf_idf_vectorizer.fit_transform(corpus).todense()
df = pd.DataFrame(tf_idf_rep)
df.columns = tf_idf_vectorizer.get_feature_names_out()
df.index = corpus
display(df)
```

Output

```
age       and  \
it was the best of times                            0.000000  0.000000
it was the worst of times                           0.000000  0.000000
it was the age of wisdom and the age of foolish...  0.617558  0.308779
best  foolishness  \
it was the best of times                            0.579897     0.000000
it was the worst of times                           0.000000     0.000000
it was the age of wisdom and the age of foolish...  0.000000     0.308779
it        of  \
it was the best of times                            0.342496  0.342496
it was the worst of times                           0.342496  0.342496
it was the age of wisdom and the age of foolish...  0.182370  0.364740
the     times  \
it was the best of times                            0.342496  0.441027
it was the worst of times                           0.342496  0.441027
it was the age of wisdom and the age of foolish...  0.364740  0.000000
was    wisdom  \
it was the best of times                            0.342496  0.000000
it was the worst of times                           0.342496  0.000000
it was the age of wisdom and the age of foolish...  0.182370  0.308779
worst
it was the best of times                            0.000000
it was the worst of times                           0.579897
it was the age of wisdom and the age of foolish...  0.000000
```

* The results from custom implementation and from scikit-learn of TFIDF are as TFIDF of scikit-learn uses
.

* Similar to CountVectorizer, TfidfVectorizer also has many , like , setting etc.

How is TF-IDF different from BOW?

Lets understand them better with a common example

Code

```
# sample data
corpus = [
"it was the best of times",
"it was the worst of times",
"it was the age of wisdom and the age of foolishness"
]
# BOW
bow_representation = get_bow_representation(corpus, True)
# Tf-IDF
tf_idf_representation = get_tf_idf(corpus)
# Group their labels
grouped_representation = []
for i in range(3):
bow_sentence = bow_representation[i]
tf_idf_sentence = tf_idf_representation[i]
grouped_sentence = {}
for word in bow_sentence.keys():
# Rounding decimals to 3 significant digits
grouped_sentence[word] = (round(bow_sentence[word], 3), round(tf_idf_sentence[word],
3))
grouped_representation.append(grouped_sentence)
df = pd.DataFrame(grouped_representation)
df.index = corpus
display(df)
```

Output

```
wisdom        best  \
it was the best of times                            (0, 0.0)  (1, 0.183)
it was the worst of times                           (0, 0.0)    (0, 0.0)
it was the age of wisdom and the age of foolish...  (1, 0.1)    (0, 0.0)
of       age  \
it was the best of times                            (1, 0.0)  (0, 0.0)
it was the worst of times                           (1, 0.0)  (0, 0.0)
it was the age of wisdom and the age of foolish...  (2, 0.0)  (2, 0.2)
worst       times  \
it was the best of times                              (0, 0.0)  (1, 0.068)
it was the worst of times                           (1, 0.183)  (1, 0.068)
it was the age of wisdom and the age of foolish...    (0, 0.0)    (0, 0.0)
was       and  \
it was the best of times                            (1, 0.0)  (0, 0.0)
it was the worst of times                           (1, 0.0)  (0, 0.0)
it was the age of wisdom and the age of foolish...  (1, 0.0)  (1, 0.1)
it foolishness  \
it was the best of times                            (1, 0.0)    (0, 0.0)
it was the worst of times                           (1, 0.0)    (0, 0.0)
it was the age of wisdom and the age of foolish...  (1, 0.0)    (1, 0.1)
the
it was the best of times                            (1, 0.0)
it was the worst of times                           (1, 0.0)
it was the age of wisdom and the age of foolish...  (2, 0.0)
```

We see that...

TF-IDF

* It computes the feature importances of a word in a document of a corpus

* It can detect and nullify the effect of stopwords on feature vector of a sentence

Bag Of Words

* It computes the frequencies (or) presence of a word in a document of a corpus

* It assigns equal priority to every word. It cannot detect stopwords

Ngram parameter (we will study ngram later on)

* and how it can be used. For example, the word tend to come in pair. So we should tokenize such words together as a single token .

* has inbuild parameters to achieve this:

Code

```
# Bigram using TfidfVectorizer
tf_idf_vectorizer = TfidfVectorizer(ngram_range=(1,2))
tf_idf_rep = tf_idf_vectorizer.fit_transform(corpus).todense()
df = pd.DataFrame(tf_idf_rep)
df.columns = tf_idf_vectorizer.get_feature_names_out()
df.index = corpus
display(df)
```

Output

```
age    age of  \
it was the best of times                             0.000000  0.000000
it was the worst of times                            0.000000  0.000000
it was the age of wisdom and the age of foolish...   0.415353  0.415353
and   and the  \
it was the best of times                             0.000000  0.000000
it was the worst of times                            0.000000  0.000000
it was the age of wisdom and the age of foolish...   0.207677  0.207677
best   best of  \
it was the best of times                             0.400008  0.400008
it was the worst of times                            0.000000  0.000000
it was the age of wisdom and the age of foolish...   0.000000  0.000000
foolishness        it  \
it was the best of times                                0.000000  0.236251
it was the worst of times                               0.000000  0.236251
it was the age of wisdom and the age of foolish...      0.207677  0.122657
it was        of  \
it was the best of times                             0.236251  0.236251
it was the worst of times                            0.236251  0.236251
it was the age of wisdom and the age of foolish...   0.122657  0.245314
of foolishness  of times  \
it was the best of times                                   0.000000  0.304216
it was the worst of times                                  0.000000  0.304216
it was the age of wisdom and the age of foolish...         0.207677  0.000000
of wisdom       the  \
it was the best of times                              0.000000  0.236251
it was the worst of times                             0.000000  0.236251
it was the age of wisdom and the age of foolish...    0.207677  0.245314
the age  the best  \
it was the best of times                             0.000000  0.400008
it was the worst of times                            0.000000  0.000000
it was the age of wisdom and the age of foolish...   0.415353  0.000000
the worst     times  \
it was the best of times                              0.000000  0.304216
it was the worst of times                             0.400008  0.304216
it was the age of wisdom and the age of foolish...    0.000000  0.000000
```

[Output continues below...]

```
was   was the  \
it was the best of times                            0.236251  0.236251
it was the worst of times                           0.236251  0.236251
it was the age of wisdom and the age of foolish...  0.122657  0.122657
wisdom  wisdom and  \
it was the best of times                            0.000000    0.000000
it was the worst of times                           0.000000    0.000000
it was the age of wisdom and the age of foolish...  0.207677    0.207677
worst  worst of
it was the best of times                            0.000000  0.000000
it was the worst of times                           0.400008  0.400008
it was the age of wisdom and the age of foolish...  0.000000  0.000000
```

Some of the drawbacks of both BOW and TFIDF method are -
1. For a large corpus, the vocabulary can be huge, hence computation and storage of both BOW and TFIDF can be difficult.
2. Both BOW and TFIDF only considers the presence or absence of words. None of them considers the meaning.

> *For example, consider the words good, nice and bad in vocabulary. All three of them would be equally different from each other according to the BOW representation, however we know that good and nice are similar in meaning and are both different from the word bad.*

Back to our task at hand...

Euclidean Distance vs Cosine Similarity

How do we calculate the distance between the words (or how do we measure the difference between the two words) ?

* Each of the word above is representend as a vector of numbers.

* So, if we could find , it would technically (using the current vectorization technique).

So the next question is

> *How do we calculate the distance between two vectors ?*

* We can imagine each of vectors above as a point in N (here 3) dimensional space.

* There are 2 popular distance metrics that we usually use to calculate the distance between any 2 points in space -

1. Euclidean Distance

This is the well known method to calculate the distance bewteen any 2 points in space. It is like .

Here,
* X and Y are the 2 vectors.
* Size of each vector is N

2. Cosine Distance

* Before we look at cosine distance, let us look at cosine similarity, because that is what we usually see being used at most places.

* Cosine similarity measures the similarity between two vectors, and is .

* As the definition suggests, cosine similarity only depends on the angle between the vectors and not the magnitude between the vectors.

Here,
* X and Y are the 2 vectors.
* θ is the angle between X and Y.

As per above formulation, $cosine \hspace{0.1cm} similarity$ can range from -1 to 1. Two vectors which lie on the same line ($\theta = 0$) have a cosine similarity of 1 and two vectors which lie in opposite direction to each other ($\theta = 180$) have a cosine similarity of -1.

Cosine distance is calculated as -

* Cosine distance can range from 0 to 2.

euclidean-distance-vs-cosine-similarity

.

Cosine vs. Euclidean - which one to use ?

* Cosine Similarity is typically used in the cases when the magnitude of the vector does not matter.

* For example:

> *In the cases of textual representation using something like BOW, a word like cricket might be present 50 times in one document which has 1000 words, and 5 times in another document with 100 words. It we calculate the distance using euclidean metric, we might not find the two documents very similar. But this is because of the difference in the length between the 2 documents. Cosine similarity accounts for this very well as this does not look at the magnitude but the angle between the documents (represented by vectors). And here, even though the difference in magnitude is high, the angle will be small.*

Coming back to the original problem -

* Given the 3 words (king, man and water), all represented using BOW vectors, lets try and find the distance between them using cosine distance metric.

Code

```
# 3 words represented by BOW vectors
king = [1, 0, 0]
man = [0, 1, 0]
water = [0, 0, 1]
# calculate cosine distance
def cosine_distance(x, y):
return 1 - np.dot(x, y) / (np.sqrt(np.dot(x, x)) * np.sqrt(np.dot(y, y)))
print("Cosine Distance")
print("Between {} and {} : {}".format("king", "man", cosine_distance(king, man)))
print("Between {} and {} : {}".format("king", "water", cosine_distance(king, water)))
print("Between {} and {} : {}".format("water", "man", cosine_distance(man, water)))
```

Output

```
Cosine Distance
Between king and man : 1.0
Between king and water : 1.0
Between water and man : 1.0
```

* The above image shows that the 3 vectors lie perpendicular to each other.

* Hence cosine similarity b/w any 2 of them is 0 (hence is distance between any 2 of them is 1)

- From our knowledge of engligh language we do know that a "king" and "man" are more similar to each other as compared to "king" and "water" (or "water" and "man").

* Example 1:
> Suppose you have two questions:
> 1. Where are you heading?
> 2. Where are you from?.

* Note: that these sentences have identical words, except for the last ones. However, they both have a different meaning.

> *Example 2:*
> *Suppose you have two questions:*

1. What is your age?
2. How old are you?.

* Note: words are completely different but both sentences mean the same.

* So this shows that BOW based vectorization technique do not capture the meaning of the words.

Preprocessing:

We would first need to process the data that we have using the techniques we discussed earlier in the lecture -

1. Split the article into sentences.

2. For each sentence in the corpus -
Convert the sentence to lowercaseExpand contractionsLemmatizationRemove stopwordsRemove punctuations

Code

```
def process_sentence(sentence, nlp_object):
# Convert to lowercase
sentence = sentence.lower()
# Exapnding contractions
sentence = contractions.fix(sentence)
# Lemmatization and removing stopwords
doc = nlp_object(sentence)
sentence = " ".join([token.lemma_ for token in doc if not token.is_stop])
# Remove punctuation
for p in string.punctuation:
sentence = sentence.replace(p, " ")
sentence = re.sub(r"\s+", " ", sentence) # Replace all whitespace characters with space
return sentence
```

Code

```
from tqdm.notebook import tqdm
# tqdm to see real time progress
tqdm.pandas()
nlp = spacy.load('en_core_web_sm') # English pipeline optimized for CPU
def process_article(article_text, nlp_object):
processed_article_sentences = []
# using nltk sentence tokenizer
for sentence in sent_tokenize(article_text):
# preprocessing each sentence using our process_sentence function
processed_article_sentences.append(process_sentence(sentence, nlp_object))
# joining preprocessed sentence as a complete paragrams of the article
return " ".join(processed_article_sentences)
articles["processed_text"] = articles["text"].progress_apply(lambda x : process_article(x,
nlp))
```

Output

```
0%|          | 0/208 [00:00<?, ?it/s]
```

At this point we have processed and tokenized the article text. Now finally let us try to find out the similarity between the articles.

How will we find similarity between documents ?

Document Similarity

Well, we would first need to make a vector representation of the documents. We have already seen a couple of ways to do that -
1. BOW
2. TF-IDF

And the similarity metric would again be cosine similarity.

Similarity using BOW

Code

```
# BOW representation of the dataset Using CountVectorizer from scikit-learn
count_vectorizer = CountVectorizer(min_df=5)
# min_df: ignore terms that have a document frequency strictly lower than the given threshold.
# Learn the vocabulary dictionary and return document-term matrix
bow_features = count_vectorizer.fit_transform(articles["processed_text"]).todense() #
todense() returns a matrix
# create dataframe
bow_features_df = pd.DataFrame(bow_features)
bow_features_df.columns = count_vectorizer.get_feature_names_out() # Get output feature names
for dataframe columns.
bow_features_df["TITLE"] = articles["title"]
bow_features_df["ID"] = articles["id"]
display(bow_features_df)
```

Output

```
00  000  01  05  06  07  10  100  1000  101  10k  10x  11  12  120  125  \
0     0    0   0   0   0   0   0     0     0    0    0   0   0   0    0
1     0    0   0   0   0   0   0     0     0    0    0   0   0   0    0
2     0    0   0   0   1   0   1     0     0    0    0   0   0   0    0
3     0    0   0   0   0   0   0     0     0    0    0   0   1   0    0
4     0    0   0   0   0   0   0     0     0    0    0   0   0   0    0
..   ..  ...  ..  ..  ..  ..  ..   ...   ...  ...  ...  ...  ..  ..  ...  ...
203   1    0   0   0   0   0   1     0     0    0    0   0   0   1    0    0
204   0    0   0   0   0   0   0     0     0    0    0   0   0   1    0    0
205   0    0   0   0   0   0   1     0     0    0    0   0   1   1    0    0
206   0    0   0   0   0   0   5     0     0    0    0   0   0   0    0    0
207   0    0   0   0   0   0   1     0     0    0    0   0   0   0    0    0
128  13  14  15  150  16  17  18  19  1980  1989  1990  1k  1st  20  200  \
0      0   0   0    0   0   0   0   0     0     0     0   0    0   0    0
1      0   0   0    0   0   0   0   0     0     0     0   0    0   0    0
2      0   0   0    0   0   0   0   0     0     0     0   0    0   0    0
3      0   0   1    1   0   0   0   0     0     0     0   0    0   0    0
4      0   0   0    0   0   0   0   0     0     0     0   0    0   0    0
..   ...  ..  ..  ..  ...  ..  ..  ..  ..   ...   ...   ...  ..  ...  ..  ...
203    0   0   0    0   0   0   0   0     0     0     0   0    0   1    0
204    0   0   1    0   0   0   0   0     0     0     0   0    0   0    0
205    0   0   0    0   0   0   0   0     0     0     0   0    0   0    0
206    0   0   0    3   0   0   0   2     0     0     0   0    0   0    0
207    0   0   0    0   0   0   0   0     0     0     0   0    0   1    0
2000  2001  2005  2008  2009  2010  2011  2012  2013  2014  2015  2016  \
0       0     0     0     0     0     0     0     0     0     0     0
1       0     0     0     0     0     0     0     0     0     0     0
2       0     0     0     0     0     0     0     0     0     0     0
3       0     0     0     0     0     0     0     0     0     0     0
4       0     0     0     0     0     0     0     0     0     0     0
..    ...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...   ...
203     1     0     0     1     0     0     0     0     0     0     0     0
204     0     0     0     0     0     0     0     0     0     0     0     0
205     0     0     0     0     0     0     0     0     0     0     0     0
206     0     0     0     0     0     0     0     0     0     0     0     0
207     0     0     0     0     0     0     0     0     0     0     0     0
2017  2018  2019  2020  2021  21  ...  winter  wire  wise  wish  \
0       0     0     0     0     0   0  ...       0     0     0     0
1       0     0     0     0     0   0  ...       0     0     0     0
2       0     0     0     0     1   0  ...       0     0     0     0
3       0     0     0     0     0   0  ...       0     0     0     0
4       1     0     1     0     0   0  ...       0     0     0     1
```

[Output continues below...]

```
..    ...   ...   ...   ...   ...  ..  ...     ...   ...   ...   ...
203     0     1     0     0     0   0  ...       0     0     0     0
204     0     0     0     0     0   0  ...       0     0     0     0
205     0     0     0     0     0   0  ...       0     0     0     0
206     1     0     0     0     0   0  ...       0     0     0     0
207     0     0     0     0     0   0  ...       0     0     0     0
withdraw  woman  wonder  wonderful  wood  word  work  worker  workflow  \
0            0       0           0     0     0     2       2         0
1            0       0           0     0     0     0       0         0
2            0       0           0     0     0     0       4         0
3            0       0           0     0     0     0       4         0
4            0       0           0     0     0     1       2         0
..         ...     ...         ...   ...   ...   ...     ...       ...
203          0       0           1     0     0     0       4         0
204          0       0           1     0     0     0       4         0
205          0       0           0     0     0     0       0         0
206          0       0           0     0     0     0      10         0
207          0       5           0     0     0     3       2         0
working  workout  world  worried  worry  worsen  worth  wow  wrap  write  \
0          0        0      0        0      0       0      0    0     0      4
1          0        0      0        0      0       0      0    0     0      1
2          0        0      0        0      0       0      0    0     0      2
3          0        0      3        0      1       0      0    0     0      4
4          0        0      0        0      0       0      0    0     0      0
..       ...      ...    ...      ...    ...     ...    ...  ...   ...    ...
203        0        0      0        0      0       1      0    0     0      1
204        0        0      0        0      0       0      0    0     0      0
205        0        0      0        0      0       0      0    0     0      0
206        0        0      1        0      0       0      0    0     0      3
207        0        0      0        1      0       0      0    0     0      2
writer  writing  wrong  www  xgboost  xi  yeah  year  yearly  yellow  \
0         0        0      0    0        1   0     0     0       0       0
1         0        0      0    1        0   0     0     0       0       0
2         0        0      1    1        0   0     0     1       0       0
3         0        0      0    0        0   0     0     0       0       0
4         0        0      0    0        0   0     0     0       0       0
..      ...      ...    ...  ...      ...  ..   ...   ...     ...     ...
203       0        0      3    0        0   0     0     4       0       0
204       0        0      2    3        0   0     0     1       0       0
205       0        0      0    0        0   0     0     0       0       0
206       0        0      0    1        0   0     0     2       0       0
207       0        0      1    0        0   0     0     1       0       0
```

[Output continues below...]

```
yes  yesterday  yield  yo  york  you  young  youtube  zero  zhou  zip  \
0      0          0      0   0     0    0      0        0     0     0    0
1      0          0      0   0     0    0      0        0     0     0    0
2      0          0      0   0     0    0      0        0     0     0    0
3      1          0      0   0     0    0      0        0     1     0    0
4      0          0      0   0     0    0      0        0     0     0    0
..   ...        ...    ...  ..   ...  ...    ...      ...   ...   ...  ...
203    0          0      0   0     0    0      1        0     0     0    0
204    1          0      0   0     0    1      0        0     0     0    0
205    0          0      0   0     0    0      0        0     1     0    0
206    0          0      0   0     0    0      0        0     0     0    0
207    2          0      0   2     0    0      0        0     0     0    0
zombie  zone  zoom  zuckerberg  \
0         0     0     0           0
1         0     0     0           0
2         0     0     0           0
3         0     0     0           0
4         0     0     0           0
..      ...   ...   ...         ...
203       0     0     0           0
204       0     2     0           0
205       0     0     0           0
206       1     1     0           0
207       0     0     0           0
TITLE   ID
0     Ensemble methods: bagging, boosting and stacking    1
1                         Understanding AUC - ROC Curve    2
2    How to work with object detection datasets in ...    3
3    11 Dimensionality reduction techniques you sho...    4
4                           The Time Series Transformer    5
..                                                   ...  ...
203    Type 2 Diabetes Reversal  The Quick Start Guide  210
204             How a 22 Day Water Fast Changed My Life  211
205                                 Breaking Your Fast  212
206           11 Unusual Tips for How to Wake Up Early  213
207  The 3 Biggest Mistakes Women Make On The Ketog...  214
[208 rows x 3709 columns]
```

How will we visualize this high dimensional data?

[T-SNE](https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html)

* T-distributed Stochastic Neighbor Embedding is a method of visualizing high dimensional data

* It converts similarities to joint probabilities and minimizes the Kullback-Leibler divergence between the joint probabilities of the low-dimensional embedding and the high-dimensional data

Code

```
from sklearn.manifold import TSNE
# using t-sne to observe any trends, and clusters.
tsne = TSNE(n_components=2) # n_components: estimated number of components
tsne_bow_features =
tsne.fit_transform(bow_features_df[count_vectorizer.get_feature_names_out()].values)
tsne_bow_features_df = pd.DataFrame(tsne_bow_features)
tsne_bow_features_df.columns = ["C1", "C2"]
tsne_bow_features_df["TITLE"] = bow_features_df["TITLE"]
tsne_bow_features_df["ID"] = bow_features_df["ID"]
display(tsne_bow_features_df)
```

Output

```
C1        C2                                              TITLE  \
0    0.253000 -4.832599   Ensemble methods: bagging, boosting and stacking
1    0.140660 -0.268687                      Understanding AUC - ROC Curve
2   -0.559838 -3.454881  How to work with object detection datasets in ...
3   -1.712352 -4.527920  11 Dimensionality reduction techniques you sho...
4    0.441553 -2.402554                        The Time Series Transformer
..        ...       ...                                               ...
203  4.747480  2.049826    Type 2 Diabetes Reversal  The Quick Start Guide
204  3.458541  2.264507            How a 22 Day Water Fast Changed My Life
205  3.477875  2.096765                                 Breaking Your Fast
206  1.979719  3.158341           11 Unusual Tips for How to Wake Up Early
207  4.600616  3.190689  The 3 Biggest Mistakes Women Make On The Ketog...
ID
0      1
1      2
2      3
3      4
4      5
..   ...
203  210
204  211
205  212
206  213
207  214
[208 rows x 4 columns]
```

Code

```
import plotly.express as px
# scatter plot of t-sne for the BOW representation
title = "T-distributed Stochastic Neighbor Embedding for BOW document representation"
fig = px.scatter(tsne_bow_features_df, x="C1", y="C2", hover_data=['TITLE'], title=title)
fig.show()
```

Output


```
if (window.MathJax && window.MathJax.Hub && window.MathJax.Hub.Config)
{window.MathJax.Hub.Config({SVG: {font: "STIX-Web"}});}               window.PlotlyConfig =
{MathJaxConfig: 'local'};
window.PLOTLYENV=window.PLOTLYENV || {};                                    if
(document.getElementById("44f50000-9a29-4d1e-bb37-08e6d9573791")) {
Plotly.newPlot(                        "44f50000-9a29-4d1e-bb37-08e6d9573791",
[{"customdata":[["Ensemble methods: bagging, boosting and stacking"],["Understanding AUC - ROC
Curve"],["How to work with object detection datasets in COCO format"],["11 Dimensionality
reduction techniques you should know in 2021"],["The Time Series Transformer"],["Learning a
Personalized Homepage"],["6 Data Science Certificates To Level Up Your Career"],["Transformers
Explained Visually (Part 2): How it works, step-by-step"],["60 Python Projects with Source
Code"],["Geometric foundations of Deep Learning"],["Machine Learning Basics with the K-Nearest
Neighbors Algorithm"],["Building RNN, LSTM, and GRU for time series using
PyTorch"],["Algorithms of the Mind"],["4 Reasons Why Economists Make Great Data Scientists
(And Why No One Tells Them)"],["How To Create A Chatbot with Python & Deep Learning In Less
Than An Hour"],["Machine Learning is Fun Part 5: Language Translation with Deep Learning and
the Magic of Sequences"],["Illustrated Guide to LSTM's and GRU's: A step by step
explanation"],["How to go from a Python newbie to a Google Certified TensorFlow Developer
under two months"],["17 Clustering Algorithms Used In Data Science and Mining"],["Introduction
to Genetic Algorithms  Including Example Code"],["Photoreal Roman Emperor
Project"],["Fundamental Techniques of Feature Engineering for Machine Learning"],["How I Got a
Job at DeepMind as a Research Engineer (without a Machine Learning Degree!)"],["Towards the
end of deep learning and the beginning of AGI"],["Tutorial: Document Classification using
WEKA"],["Understanding Random Forest"],["Probability concepts explained: Maximum likelihood
estimation"],["Transformers Explained Visually (Part 3): Multi-head Attention, deep
dive"],["180 Data Science and Machine Learning Projects with Python"],["5 Online Courses I
Took as a Self-Taught Data Scientist"],["Machine Learning is Fun! Part 4: Modern Face
Recognition with Deep Learning"],["9 Distance Measures in Data Science"],["The Sexiest Job of
the 21st Century Isn't \"Sexy\" Anymore"],["Machine Learning is Fun! Part 3: Deep Learning and
Convolutional Neural Networks"],["A Comprehensive Guide to Convolutional Neural Networks  the
```

ELI5 way"],["20 Necessary Requirements of a Perfect Laptop for Data Science and Machine Learning Tasks"],["3 Beginner Mistakes I've Made in My Data Science Career"],["Understanding Variational Autoencoders (VAEs)"],["Setting Up a New M1 MacBook for Data Science"],["Are The New M1 Macbooks Any Good for Data Science? Let's Find Out"],["Simple and Multiple Linear Regression in Python"],["I tripled my income with data science. Here's how."],["Keyword Extraction process in Python with Natural Language Processing(NLP)"],["Machine learning models for 100% better returns in Algo-trading"],["How to Install Ubuntu Desktop With a Graphical User Interface in WSL2"],["Understanding Contrastive Learning"],["Understanding Semantic Segmentation with UNET"],["How Transformers Work"],["Yes you should understand backprop"],["PCA using Python (scikit-learn)"],["Hyperparameter Tuning the Random Forest in Python"],["Top 3 Reasons Why I Sold My M1 Macbook Pro as a Data Scientist"],["TRAIN A CUSTOM YOLOv4 OBJECT DETECTOR (Using Google Colab)"],["6 Machine Learning Certificates to Pursue in 2021"],["What to do with \"small\" data?"],["Time Series Forecasting with PyCaret Regression Module"],["15 Habits I Learned from Highly Effective Data Scientists"],["OVER 100 Data Scientist Interview Questions and Answers!"],["Activation Functions in Neural Networks"],["A One-Stop Shop for Principal Component Analysis"],["5 Things You Should Know About Covariance"],["How to build your own Neural Network from scratch in Python"],["The best explanation of Convolutional Neural Networks on the Internet!"],["Lambda Functions with Practical Examples in Python"],["The Complete Guide to Time Series Analysis and Forecasting"],["8 Ways to Filter Pandas Dataframes"],["Standing with Dr. Timnit Gebru #ISupportTimnit #BelieveBlackWomen"],["Visualising high-dimensional datasets using PCA and t-SNE in Python"],["How to Study for the Google Data Analytics Professional Certificate"],["Is Data Science Still a Rising Career in 2021"],["Simple Reinforcement Learning with Tensorflow Part 0: Q-Learning with Tables and Neural Networks"],["How To Grow From Non-Coder to Data Scientist in 6 Months"],["Apple's New M1 Chip is a Machine Learning Beast"],["Train\u002fTest Split and Cross Validation in Python"],["Text Classification with NLP: Tf-Idf vs Word2Vec vs BERT"],["The 5 Clustering Algorithms Data Scientists Need to Know"],["Building A Logistic Regression in Python, Step by Step"],["Springer has released 65 Machine Learning and Data books for free"],["Every single Machine Learning course on the internet, ranked by your reviews"],["TensorFlow Tutorial Part 1"],["The 7 Best Data Science and Machine Learning

Podcasts"],["Random Forest in Python"],["How to Land a Data Analytics Job in 6 Months"],["What is a Transformer?"],["Fundamentals Of Statistics For Data Scientists and Analysts"],["Is Google's AI research about to implode?"],["Machine Learning is Fun! Part 2"],["Understanding Generative Adversarial Networks (GANs)"],["17 types of similarity and dissimilarity measures used in data science."],["Deep Learning Is Going to Teach Us All the Lesson of Our Lives: Jobs Are for Machines"],["I interviewed at five top companies in Silicon Valley in five days, and luckily got five job offers"],["Advantages and Disadvantages of Artificial Intelligence"],["Could Nim Replace Python?"],["An End-to-End Project on Time Series Analysis and Forecasting with Python"],["Data Scientists Will be Extinct in 10 Years"],["Import all Python libraries in one line of code"],["Enchanted Random Forest"],["Machine Learning in a Week"],["50+ Statistics Interview Questions and Answers for Data Scientists for 2022"],["Top 10 Data Science Projects for Beginners"],["Check For a Substring in a Pandas DataFrame Column"],["Machine Learning is Fun Part 6: How to do Speech Recognition with Deep Learning"],["Everything You Need to Know About Artificial Neural Networks"],["Ways to Detect and Remove the Outliers"],["How to Crush the Crypto Market, Quit Your Job, Move to Paradise and Do Whatever You Want the Rest of Your Life"],["Bitcoin Is Venice"],["How I Built a Net Worth of $500,000 Before Age 30"],["Nassim Nicholas Taleb on Self-Education and Doing the Math (Ep. 41 Live at Mercatus)"],["Public Mint Polkastarter IDO: Launching 23rd of February, Whitelist Now Open!"],["This is How I Made $40k In Passive Income By Age 26"],["Introducing The Iron Bank"],["I used Acorns, Robinhood, and Stash for 2 years. This is what I learned and earned."],["Explaining blockchain how proof of work enables trustless consensus"],["Wealthfront:Silicon Valley Tech at Wall Street Prices"],["Crypto Trading Bots A helpful guide for beginners [2020]"],["Why People Still Don't Get Cryptocurrency"],["Australia's Economy is a House of Cards"],["My First Two Months Trading Stocks with Robinhood"],["What It Takes to Go from $0 to $1 Million in Less Than One Year"],["Tesla Is Dead (And Elon Musk Knows It)"],["Uber's Credit Card Is Bankrupting Restaurants... and It's All Your Fault"],["A Quick Starter Guide to Leveraged Trading at BitMEX"],["I Made $3 Million in Crypto. These are the 26 Rules I Learned."],["The Black-Scholes formula, explained"],["You Will Never Be Rich If You Keep Doing These 10 things"],["Heroes Give, Superheroes Borrow"],["Could Bitcoin's Bull Market End Next

Month?"],["The Exact Steps I Followed to Make $1,500+ of Passive Income Every Month"],["You May Have A Poor Person's Mindset And Not Know It"],["A Definitive Guide to Why Life Is So Terrible for Most Millennials"],["The Berkshire Hathaway of The Internet"],["How to invest in Bitcoin properly. Blockchain and other cryptocurrencies"],["Machine learning in finance: Why, what & how"],["Minimum Wage Artists"],["Richart + @GoGo "],["How to store Bitcoins and other cryptocurrencies properly."],["I was wrong about Ethereum"],["Deep Learning the Stock Market"],["Financial Fridays: It's Financial Suicide To Own A House"],["Gauge Theory Does Not Fix This"],["High Frequency Trading on the Coinbase Exchange"],["A $1000 Bitcoin Investment Won't Make You Rich"],["How To Legally Own Another Person"],["5 Things NOT to Do in the Robinhood App for Stock Trading"],["The One Word That Explains Why Economics Professors Are Not Billionaires"],["Building your credit history"],["I Retired at 35 Here Are 5 Lessons from My First 6 Months of Freedom"],["Why you should never use Upwork, ever."],["Facebook Can't Be Fixed."],["A comprehensive guide to downloading stock prices in Python"],["Detecting Credit Card Fraud Using Machine Learning"],["2018 & : KOKO COMBO icash "],["Is Wealthfront Worth it?"],["How I Slowly Became A \"Middle-Class\" Millionaire"],["SPY vs. QQQ: Investing in Different Indexes"],["I Won $104 Million for Blowing the Whistle on My CompanyBut Somehow I Was the Only One Who Went to Jail"],["Robinhood Lends \"Your\" Shares to Short Sellers (and Keeps All the Proceeds)"],["The Secret to Making 2000% in Stocks Overnight, the Anavex story."],["4 Unusual Side Hustles I Do to Earn an Extra $1,500 a Month"],["Meet 'Spoofy'. How a Single entity dominates the price of Bitcoin."],["How I Started Saving a TON of Money."],["How I transformed $6,000 to $3,000,000"],["Cliff Asness on Marvel vs. DC and Why Never to Share a Gym with Cirque du Soleil (Ep. 5 Live at Mason)"],["Paypal froze our funds, then offered us a business loan"],["A Visual Explanation of Algorithmic Stablecoins"],["Algorithmic Trading Bot: Python"],["Chernobyl's Blown Up Reactor 4 Just Woke Up"],["The Unexpected Case of the Disappearing Flu"],["The Diet & Workout Plan of a Full-Time Traveling Family HIS"],["The Cult of Work You Never Meant to Join"],["5 Things to do in the First 24 Hours of a Cold or Flu"],["The Sweet Spot for Intermittent Fasting"],["The 6 Types of ICU Nurse"],["How to biohack your intelligence with everything from sex to modafinil to MDMA"],["8 Common Arguments Against Vaccines"],["A Reasonably Detailed Guide to Optimizing Your iPhone for Productivity, Focus and Your Own Health"],["The cure for type 2 diabetes is

known, but few are aware"],["Face mapping: how to deal with acne like a true detective"],["I'm 32 and spent $200k on biohacking. Became calmer, thinner, extroverted, healthier & happier."],["I need more iron"],["What I Learned From Quitting Coffee After 15 Years Of Daily Consumption"],["The forgotten art of untucking the tail"],["Hip Replacement Isn't What It Used To Be 12 Weeks Ago"],["Coronavirus: Why You Must Act Now"],["Manufacturers have been using nanotechnology-derived graphene in face masks now there are safety concerns"],["Eben Byers: The Man Who Drank Radioactive Water Until His Jaw Fell Off"],["My Intermittent Fasting Lifestyle: How I Dropped 50 Pounds"],["I Stopped Drinking for 30 Days. Here's What Happened."],["How to lose 10+ pounds of fat a month- even if you have a slow metabolism"],["If You're Buying Your Weed Solely Based on the THC Level, You're Doing it Wrong."],["2016 Is Not Killing People"],["How I Learned to Sleep Only Three Hours Per Night (and Why You Should Too)"],["I fasted for 11 days, here is what happened."],["Coronavirus: The Hammer and the Dance"],["What Happens in Your Body When You Quit Drinking"],["The Science Behind Fat Metabolism"],["The Easiest Way to Lose 125 Pounds Is to Gain 175 Pounds"],["7 things I did to reboot my life."],["How I lost 10kg in 60 days: My 7-step weight loss plan"],["The Foo Fighters' AIDS denialism should be on the record"],["The Uncut History of Male Circumcision"],["10 Things You Can Do This Morning To Heal Your Anxiety"],["I just lost 100 pounds. Here's whyalmost nobody else will!"],["Type 2 Diabetes Reversal The Quick Start Guide"],["How a 22 Day Water Fast Changed My Life"],["Breaking Your Fast"],["11 Unusual Tips for How to Wake Up Early"],["The 3 Biggest Mistakes Women Make On The Ketogenic Diet (And How To Fix
Them)"]],"hovertemplate":"C1=%{x}\u003cbr\u003eC2=%{y}\u003cbr\u003eTITLE=%{customdata[0]}\u003cextra\u003e\u003c\u002fextra\u003e","legendgroup":"","marker":{"color":"#636efa","symbol":"circle"},"mode":"markers","name":"","orientation":"v","showlegend":false,"x":[0.25299972,0.14066029,-0.5598376,-1.7123524,0.44155306,0.7244344,-3.6318078,4.612736,-0.6939202,3.277693,-1.964162,0.15622608,2.470655,-2.3816905,1.5642217,2.1521828,-4.141387,-1.7607564,-5.7718735,0.47813508,-0.39933893,0.13913007,-5.6589346,-2.165403,-0.09352872,5.584858,0.41373992,4.6290865,-0.47681993,-3.906913,2.8239932,-6.138245,-2.985446,3.9345715,3.6974032,-1.3675834,-2.5083625,0.37190175,-0.74010646,-0.853511,-0.60560876,-2.5216863,-1.457145,-3.3146484,1.8521031,1.846657,3.944199,4.5246906,0.6429291,-

1.6438873,-0.23240972,-0.6819501,1.5296054,-3.8299623,-0.11738305,-0.63348275,-3.0216742,-
1.8622898,1.4494358,-0.66256446,-0.10526606,3.4098964,3.352204,1.5936632,-
5.577258,0.39837125,-4.1466823,-1.6485761,-4.219535,-2.9563184,-3.6607304,-3.541951,-
1.0943226,-0.3846951,1.474267,-5.655846,-0.6538666,-2.8151956,-4.0022864,-0.5571646,-
1.1164402,-0.56182843,-3.400786,4.6240897,-1.8248049,1.518897,1.3452518,0.8413228,-
6.203111,2.9802265,-5.485455,2.3458009,-2.0733218,-5.4530177,-2.2647648,-1.8601112,5.5670123,-
1.3846523,-1.8315852,-3.542881,0.36446375,-0.6451847,3.5625014,-1.8818178,-0.20465976,-
1.653876,0.3567609,1.7491904,-0.4496622,-0.8963981,-4.8093224,-2.0623724,-0.86732966,-
0.9942325,-3.7811425,0.19660409,-0.15527792,-2.543013,-0.40456945,-2.0836167,-2.8310993,-
2.4203434,-1.9975122,-2.837085,0.38960537,-0.3541275,-
1.5972267,1.1924974,0.2581237,0.08767733,-0.09300044,-1.5438389,-3.4108496,0.00878801,-
0.4379407,-1.0564133,-0.9717927,1.1375408,0.10153179,-0.8644147,-3.8617792,-
1.6065834,0.20587257,-2.6637738,-0.028561229,-0.52698904,0.7932505,-3.840741,-0.90317476,-
2.4447303,-1.308548,-0.4398119,-1.1500015,-0.15964809,-1.7153778,0.15026662,-1.9648772,-
2.958478,1.4025402,-3.1751723,-0.42698154,-2.7738686,1.0654943,-0.5861532,-4.488358,-3.11916,-
0.24179801,-1.780375,2.6085322,1.9202538,-0.35608506,-
1.6264496,0.69122905,3.610976,0.93475837,3.5868716,4.316623,0.26332408,3.9051454,-
5.067452,1.8347449,-1.2087456,1.6859767,2.5759478,1.2332664,-
0.09350816,3.5225372,1.6908665,3.7905378,0.7786492,-
0.1644186,1.5066416,3.615443,2.5557263,1.454697,4.786486,2.9175391,2.5858724,3.5330799,0.18013
67,0.044533808,2.5491624,3.307612,4.74748,3.4585412,3.4778745,1.9797193,4.6006165],"xaxis":"x"
,"y":[-4.832599,-0.2686873,-3.4548807,-4.52792,-2.4025536,-1.434994,-3.275751,-
4.6821055,0.40472332,-2.3307319,-3.6386423,-4.380679,-1.4215411,-2.1249025,-2.8673651,-
4.1252494,-0.073592,-1.1092193,-3.76059,-0.0045736106,0.55812407,-3.0034091,1.1528484,-
0.4860664,-1.7275984,-0.39140454,-6.233281,-4.682409,0.45904267,-2.1457605,-1.1546618,-
2.2598312,-2.404789,-2.1332626,-1.3547384,-0.118723005,-1.8478651,-6.3419967,0.13409472,-
0.4427909,-3.086927,-1.3293225,0.0547832,2.139569,-2.8099952,-1.5522517,-1.203469,-4.6223884,-
0.31381926,-4.265336,-4.7178187,0.300834,-2.1636553,-3.7220206,-4.3467975,-3.4167535,-
2.4821332,-6.294312,-0.61608994,-2.403625,-0.8613935,-2.3253438,-1.972033,-0.67938095,-
0.64180416,-1.1774886,-4.69335,-4.0900297,-2.1059742,-2.381627,-0.056591853,-2.2789352,-

0.6576594,-4.346671,-3.40558,-3.6962333,-2.4851525,0.7305848,-4.4496064,-0.4811733,-0.886256,-5.0328937,-1.6792461,-4.7315297,-6.399737,-4.276051,-4.438035,-6.331344,-2.3941185,0.2905764,1.1039265,0.3791951,0.15649706,-0.65544176,-2.2228277,0.38852313,-0.39715424,-1.5174801,-6.1919117,-1.5779399,-1.1298543,-1.3400552,-2.3039873,-2.6641796,6.423209,6.392579,3.0870497,5.586948,0.67732465,3.5331008,2.854316,3.8960774,2.3052678,3.6480713,4.83487,5.640751,6.473507,3.6097605,1.7623299,2.587947,0.08121988,5.475244,5.4659443,2.4957018,5.124053,1.1073825,5.5342426,5.033515,4.9682803,4.1381674,2.6468987,6.1823616,-4.276932,1.5891147,0.659685,1.4333332,2.4542646,-2.7429543,3.5539258,5.027209,5.027251,5.4114704,3.9574926,3.700182,5.071859,0.81702304,3.9759316,1.0861212,1.3860646,1.4102755,-3.300483,0.6879007,3.6307871,2.8877373,2.3736863,1.9751117,1.8892413,3.4545252,0.8035454,5.86293,2.0135465,4.0014124,6.4208755,1.2051686,2.4925444,1.6425741,0.50964385,0.9338538,3.2365348,3.7337778,0.7129891,1.4320635,1.7650071,5.1704025,0.9387256,5.486603,2.9422967,0.6088593,4.5207887,3.0538733,1.3497256,0.88947356,2.5638406,7.2860436,0.21672115,1.0136712,2.1696208,2.2859478,3.2473414,1.3840153,1.0226934,3.0127113,2.0502982,7.2464547,1.6995729,3.3426657,4.5329094,4.671042,3.4075043,1.0014935,0.5911007,2.524681,4.3850937,2.0498261,2.264507,2.0967648,3.1583407,3.1906893],"yaxis":"y","type":"scatter"}],
{"template":{"data":{"histogram2dcontour":[{"type":"histogram2dcontour","colorbar":{"outlinewidth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]}],"choropleth":[{"type":"choropleth","colorbar":{"outlinewidth":0,"ticks":""}}],"histogram2d":[{"type":"histogram2d","colorbar":{"outlinewidth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]}],"heatmap":[{"type":"heatmap","colorbar":{"outlinewidth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f9

21"]]}],"heatmapgl":[{"type":"heatmapgl","colorbar":{"outlinewidth":0,"ticks":""},"colorscale"
:[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.33333333333
33333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.666666666666
6666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]
}],"contourcarpet":[{"type":"contourcarpet","colorbar":{"outlinewidth":0,"ticks":""}}],"contou
r":[{"type":"contour","colorbar":{"outlinewidth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[
0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0
.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.
7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]}],"surface":[{"typ
e":"surface","colorbar":{"outlinewidth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[0.1111111
111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.44444444
44444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.777777777
7777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]}],"mesh3d":[{"type":"mesh3d
","colorbar":{"outlinewidth":0,"ticks":""}}],"scatter":[{"fillpattern":{"fillmode":"overlay","
size":10,"solidity":0.2},"type":"scatter"}],"parcoords":[{"type":"parcoords","line":{"colorbar
":{"outlinewidth":0,"ticks":""}}}],"scatterpolargl":[{"type":"scatterpolargl","marker":{"color
bar":{"outlinewidth":0,"ticks":""}}}],"bar":[{"error_x":{"color":"#2a3f5f"},"error_y":{"color"
:"#2a3f5f"},"marker":{"line":{"color":"#E5ECF6","width":0.5},"pattern":{"fillmode":"overlay","
size":10,"solidity":0.2}},"type":"bar"}],"scattergeo":[{"type":"scattergeo","marker":{"colorba
r":{"outlinewidth":0,"ticks":""}}}],"scatterpolar":[{"type":"scatterpolar","marker":{"colorbar
":{"outlinewidth":0,"ticks":""}}}],"histogram":[{"marker":{"pattern":{"fillmode":"overlay","si
ze":10,"solidity":0.2}},"type":"histogram"}],"scattergl":[{"type":"scattergl","marker":{"color
bar":{"outlinewidth":0,"ticks":""}}}],"scatter3d":[{"type":"scatter3d","line":{"colorbar":{"ou
tlinewidth":0,"ticks":""}},"marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scattermapbo
x":[{"type":"scattermapbox","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scatterter
nary":[{"type":"scatterternary","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scatterc
arpet":[{"type":"scattercarpet","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"carpet"
:[{"aaxis":{"endlinecolor":"#2a3f5f","gridcolor":"white","linecolor":"white","minorgridcolor":
"white","startlinecolor":"#2a3f5f"},"baxis":{"endlinecolor":"#2a3f5f","gridcolor":"white","lin
ecolor":"white","minorgridcolor":"white","startlinecolor":"#2a3f5f"},"type":"carpet"}],"table"

:[{"cells":{"fill":{"color":"#EBF0F8"},"line":{"color":"white"}},"header":{"fill":{"color":"#C
8D4E3"},"line":{"color":"white"}},"type":"table"}],"barpolar":[{"marker":{"line":{"color":"#E5
ECF6","width":0.5},"pattern":{"fillmode":"overlay","size":10,"solidity":0.2}},"type":"barpolar
"}],"pie":[{"automargin":true,"type":"pie"}]},"layout":{"autotypenumbers":"strict","colorway":
["#636efa","#EF553B","#00cc96","#ab63fa","#FFA15A","#19d3f3","#FF6692","#B6E880","#FF97FF","#F
ECB52"],"font":{"color":"#2a3f5f"},"hovermode":"closest","hoverlabel":{"align":"left"},"paper_
bgcolor":"white","plot_bgcolor":"#E5ECF6","polar":{"bgcolor":"#E5ECF6","angularaxis":{"gridcol
or":"white","linecolor":"white","ticks":""},"radialaxis":{"gridcolor":"white","linecolor":"whi
te","ticks":""}},"ternary":{"bgcolor":"#E5ECF6","aaxis":{"gridcolor":"white","linecolor":"whit
e","ticks":""},"baxis":{"gridcolor":"white","linecolor":"white","ticks":""},"caxis":{"gridcolo
r":"white","linecolor":"white","ticks":""}},"coloraxis":{"colorbar":{"outlinewidth":0,"ticks":
""}},"colorscale":{"sequential":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.22222222222
22222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.555555555555
5556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888
888,"#fdca26"],[1.0,"#f0f921"]],"sequentialminus":[[0.0,"#0d0887"],[0.1111111111111111,"#46039
f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786
"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"
],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]],"diverging":[[0,"#8e0152"],[0.1,"#c51b7d"],[
0.2,"#de77ae"],[0.3,"#f1b6da"],[0.4,"#fde0ef"],[0.5,"#f7f7f7"],[0.6,"#e6f5d0"],[0.7,"#b8e186"]
,[0.8,"#7fbc41"],[0.9,"#4d9221"],[1,"#276419"]]},"xaxis":{"gridcolor":"white","linecolor":"whi
te","ticks":"","title":{"standoff":15},"zerolinecolor":"white","automargin":true,"zerolinewidt
h":2},"yaxis":{"gridcolor":"white","linecolor":"white","ticks":"","title":{"standoff":15},"zer
olinecolor":"white","automargin":true,"zerolinewidth":2},"scene":{"xaxis":{"backgroundcolor":"
#E5ECF6","gridcolor":"white","linecolor":"white","showbackground":true,"ticks":"","zerolinecol
or":"white","gridwidth":2},"yaxis":{"backgroundcolor":"#E5ECF6","gridcolor":"white","linecolor
":"white","showbackground":true,"ticks":"","zerolinecolor":"white","gridwidth":2},"zaxis":{"ba
ckgroundcolor":"#E5ECF6","gridcolor":"white","linecolor":"white","showbackground":true,"ticks"
:"","zerolinecolor":"white","gridwidth":2}},"shapedefaults":{"line":{"color":"#2a3f5f"}},"anno
tationdefaults":{"arrowcolor":"#2a3f5f","arrowhead":0,"arrowwidth":1},"geo":{"bgcolor":"white"
,"landcolor":"#E5ECF6","subunitcolor":"white","showland":true,"showlakes":true,"lakecolor":"wh

```
ite"},"title":{"x":0.05},"mapbox":{"style":"light"}}},"xaxis":{"anchor":"y","domain":[0.0,1.0]
,"title":{"text":"C1"}},"yaxis":{"anchor":"x","domain":[0.0,1.0],"title":{"text":"C2"}},"legen
d":{"tracegroupgap":0},"title":{"text":"T-distributed Stochastic Neighbor Embedding for BOW
document representation"}},                        {"responsive": true}
).then(function(){
var gd = document.getElementById('44f50000-9a29-4d1e-bb37-08e6d9573791');
var x = new MutationObserver(function (mutations, observer) {{
var display = window.getComputedStyle(gd).display;
if (!display || display === 'none') {{
console.log([gd, 'removed!']);
Plotly.purge(gd);
observer.disconnect();
}}
}});
// Listen for the removal of the full notebook cells
var notebookContainer = gd.closest('#notebook-container');
if (notebookContainer) {{
x.observe(notebookContainer, {childList: true});
}}
// Listen for the clearing of the current output cell
var outputEl = gd.closest('.output');
if (outputEl) {{
x.observe(outputEl, {childList: true});
}}
})               };
```

What do we notice in the above graph ?

* We see most of documents to the right half describe articles related to Deep learning.

* Articles plotted to the left describe Coronavirus, Economics and other miscellaneous articles

Now, let us see if we can find similar documents using BOW reresentation.

Code

```
from sklearn.metrics.pairwise import cosine_similarity
def get_similar_documents(all_article_rep_df, article_id, features):
# extracting features of a article
this_article_rep = all_article_rep_df[all_article_rep_df["ID"] == article_id][features]
other_article_rep = all_article_rep_df[all_article_rep_df["ID"] != article_id][features]
# calculating cosine similarity
similarity_matrix = cosine_similarity(this_article_rep, other_article_rep)
similar_articles = list(zip(similarity_matrix[0].tolist(),
all_article_rep_df["TITLE"].tolist()))
# sorting
similar_articles = sorted(similar_articles, key = lambda x : x[0], reverse = True)
print("Reference Article : {}".format(all_article_rep_df[all_article_rep_df["ID"] ==
article_id]["TITLE"].values[0]))
print("**** Similar Articles ****")
# top 5 similar articles
for score, title in similar_articles[:5]:
print(title)
print()
# Let us check top 5 similar articles for some of the articles in our corpus
get_similar_documents(bow_features_df, 90, count_vectorizer.get_feature_names_out())
get_similar_documents(bow_features_df, 80, count_vectorizer.get_feature_names_out())
get_similar_documents(bow_features_df, 150, count_vectorizer.get_feature_names_out())
get_similar_documents(bow_features_df, 205, count_vectorizer.get_feature_names_out())
```

Output

```
Reference Article : 17 types of similarity and dissimilarity measures used in data science.
**** Similar Articles ****
9 Distance Measures in Data Science
17 Clustering Algorithms Used In Data Science and Mining
Machine Learning Basics with the K-Nearest Neighbors Algorithm
OVER 100 Data Scientist Interview Questions and Answers!
Fundamental Techniques of Feature Engineering for Machine Learning
Reference Article : TensorFlow Tutorial Part 1
**** Similar Articles ****
How to go from a Python newbie to a Google Certified TensorFlow Developer under two months
Enchanted Random Forest
The 7 Best Data Science and Machine Learning Podcasts
PCA using Python (scikit-learn)
Time Series Forecasting with PyCaret Regression Module
Reference Article : The One Word That Explains Why Economics Professors Are Not Billionaires
**** Similar Articles ****
Why People Still Don't Get Cryptocurrency
You May Have A Poor Person's Mindset And Not Know It
You Will Never Be Rich If You Keep Doing These 10 things
How I transformed $6,000 to $3,000,000
The Exact Steps I Followed to Make $1,500+ of Passive Income Every Month
Reference Article : How I lost 10kg in 60 days: My 7-step weight loss plan
**** Similar Articles ****
10 Things You Can Do This Morning To Heal Your Anxiety
How to lose 10+ pounds of fat a month- even if you have a slow metabolism
The Diet & Workout Plan of a Full-Time Traveling Family  HIS
The cure for type 2 diabetes is known, but few are aware
The Easiest Way to Lose 125 Pounds Is to Gain 175 Pounds
```

Result:
- The suggested articles do seem to be similar to those in reference.
- BOW looks for presence and absence of words in different documents, so documents which have similar words would also have similar BOW representation, and hence would be more similar.

Similarity using TF-IDF

Code

```
# TF-IDF representation of the dataset Using TfidfVectorizer from scikit-learn
tfidf_vectorizer = TfidfVectorizer(min_df=5)
# min_df: ignore terms that have a document frequency strictly lower than the given threshold.
tfidf_features = tfidf_vectorizer.fit_transform(articles["processed_text"]).todense()  #
todense() returns a matrix
# create dataframe
tfidf_features_df = pd.DataFrame(tfidf_features)
tfidf_features_df.columns = tfidf_vectorizer.get_feature_names_out()  # Get output feature
names for dataframe columns.
tfidf_features_df["TITLE"] = articles["title"]
tfidf_features_df["ID"] = articles["id"]
display(tfidf_features_df)
```

Output

```
00  000   01   05        06   07        10  100  1000  101  10k  \
0    0.000000  0.0  0.0  0.0  0.000000  0.0  0.000000  0.0   0.0  0.0  0.0
1    0.000000  0.0  0.0  0.0  0.000000  0.0  0.000000  0.0   0.0  0.0  0.0
2    0.000000  0.0  0.0  0.0  0.019288  0.0  0.008322  0.0   0.0  0.0  0.0
3    0.000000  0.0  0.0  0.0  0.000000  0.0  0.000000  0.0   0.0  0.0  0.0
4    0.000000  0.0  0.0  0.0  0.000000  0.0  0.000000  0.0   0.0  0.0  0.0
..        ...  ...  ...  ...       ...  ...       ...  ...   ...  ...  ...
203  0.015118  0.0  0.0  0.0  0.000000  0.0  0.006343  0.0   0.0  0.0  0.0
204  0.000000  0.0  0.0  0.0  0.000000  0.0  0.000000  0.0   0.0  0.0  0.0
205  0.000000  0.0  0.0  0.0  0.000000  0.0  0.016279  0.0   0.0  0.0  0.0
206  0.000000  0.0  0.0  0.0  0.000000  0.0  0.043596  0.0   0.0  0.0  0.0
207  0.000000  0.0  0.0  0.0  0.000000  0.0  0.007900  0.0   0.0  0.0  0.0
10x        11        12  120  125  128   13        14        15  150  \
0    0.0  0.000000  0.000000  0.0  0.0  0.0  0.0  0.000000  0.000000  0.0
1    0.0  0.000000  0.000000  0.0  0.0  0.0  0.0  0.000000  0.000000  0.0
2    0.0  0.000000  0.000000  0.0  0.0  0.0  0.0  0.000000  0.000000  0.0
3    0.0  0.006738  0.000000  0.0  0.0  0.0  0.0  0.008670  0.006796  0.0
4    0.0  0.000000  0.000000  0.0  0.0  0.0  0.0  0.000000  0.000000  0.0
..   ...       ...       ...  ...  ...  ...  ...       ...       ...  ...
203  0.0  0.000000  0.008093  0.0  0.0  0.0  0.0  0.000000  0.000000  0.0
204  0.0  0.000000  0.019866  0.0  0.0  0.0  0.0  0.027918  0.000000  0.0
205  0.0  0.022684  0.020770  0.0  0.0  0.0  0.0  0.000000  0.000000  0.0
206  0.0  0.000000  0.000000  0.0  0.0  0.0  0.0  0.000000  0.036765  0.0
207  0.0  0.000000  0.000000  0.0  0.0  0.0  0.0  0.000000  0.000000  0.0
16   17        18   19  1980  1989  1990   1k  1st        20  200  \
0    0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
1    0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
2    0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
3    0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
4    0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
..   ...  ...       ...  ...   ...   ...   ...  ...  ...       ...  ...
203  0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.007529  0.0
204  0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
205  0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
206  0.0  0.0  0.031268  0.0   0.0   0.0   0.0  0.0  0.0  0.000000  0.0
207  0.0  0.0  0.000000  0.0   0.0   0.0   0.0  0.0  0.0  0.009377  0.0
2000  2001  2005      2008  2009  2010  2011  2012  2013  2014  2015  \
0    0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
1    0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
2    0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
3    0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
4    0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
```

[Output continues below...]

```
..        ...   ...   ...       ...   ...   ...   ...   ...   ...   ...   ...
203  0.012889   0.0   0.0  0.013989   0.0   0.0   0.0   0.0   0.0   0.0   0.0
204  0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
205  0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
206  0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
207  0.000000   0.0   0.0  0.000000   0.0   0.0   0.0   0.0   0.0   0.0   0.0
2016      2017      2018      2019  2020      2021   21  ...  winter  \
0     0.0  0.000000  0.000000  0.000000   0.0  0.000000  0.0  ...     0.0
1     0.0  0.000000  0.000000  0.000000   0.0  0.000000  0.0  ...     0.0
2     0.0  0.000000  0.000000  0.000000   0.0  0.014724  0.0  ...     0.0
3     0.0  0.000000  0.000000  0.000000   0.0  0.000000  0.0  ...     0.0
4     0.0  0.034284  0.000000  0.039491   0.0  0.000000  0.0  ...     0.0
..    ...       ...       ...       ...   ...       ...  ...  ...     ...
203   0.0  0.000000  0.011077  0.000000   0.0  0.000000  0.0  ...     0.0
204   0.0  0.000000  0.000000  0.000000   0.0  0.000000  0.0  ...     0.0
205   0.0  0.000000  0.000000  0.000000   0.0  0.000000  0.0  ...     0.0
206   0.0  0.014853  0.000000  0.000000   0.0  0.000000  0.0  ...     0.0
207   0.0  0.000000  0.000000  0.000000   0.0  0.000000  0.0  ...     0.0
wire  wise      wish  withdraw     woman    wonder  wonderful  wood  \
0     0.0   0.0  0.000000       0.0  0.000000  0.000000        0.0   0.0
1     0.0   0.0  0.000000       0.0  0.000000  0.000000        0.0   0.0
2     0.0   0.0  0.000000       0.0  0.000000  0.000000        0.0   0.0
3     0.0   0.0  0.000000       0.0  0.000000  0.000000        0.0   0.0
4     0.0   0.0  0.034284       0.0  0.000000  0.000000        0.0   0.0
..    ...   ...       ...       ...       ...       ...        ...   ...
203   0.0   0.0  0.000000       0.0  0.000000  0.010314        0.0   0.0
204   0.0   0.0  0.000000       0.0  0.000000  0.025320        0.0   0.0
205   0.0   0.0  0.000000       0.0  0.000000  0.000000        0.0   0.0
206   0.0   0.0  0.000000       0.0  0.000000  0.000000        0.0   0.0
207   0.0   0.0  0.000000       0.0  0.072839  0.000000        0.0   0.0
word      work  worker  workflow  working  workout     world  \
0    0.008056  0.005185     0.0       0.0      0.0       0.0  0.000000
1    0.000000  0.000000     0.0       0.0      0.0       0.0  0.000000
2    0.000000  0.022901     0.0       0.0      0.0       0.0  0.000000
3    0.000000  0.013306     0.0       0.0      0.0       0.0  0.013772
4    0.021512  0.027692     0.0       0.0      0.0       0.0  0.000000
..        ...       ...     ...       ...      ...       ...       ...
203  0.000000  0.017453     0.0       0.0      0.0       0.0  0.000000
204  0.000000  0.042845     0.0       0.0      0.0       0.0  0.000000
205  0.000000  0.000000     0.0       0.0      0.0       0.0  0.000000
206  0.000000  0.059983     0.0       0.0      0.0       0.0  0.008278
207  0.025332  0.010870     0.0       0.0      0.0       0.0  0.000000
```

[Output continues below...]

```
worried     worry    worsen  worth  wow  wrap     write  writer  \
0    0.000000  0.000000  0.000000    0.0  0.0   0.0  0.013613     0.0
1    0.000000  0.000000  0.000000    0.0  0.0   0.0  0.014092     0.0
2    0.000000  0.000000  0.000000    0.0  0.0   0.0  0.015030     0.0
3    0.000000  0.008339  0.000000    0.0  0.0   0.0  0.017465     0.0
4    0.000000  0.000000  0.000000    0.0  0.0   0.0  0.000000     0.0
..        ...       ...       ...    ...  ...   ...       ...     ...
203  0.000000  0.000000  0.015592    0.0  0.0   0.0  0.005727     0.0
204  0.000000  0.000000  0.000000    0.0  0.0   0.0  0.000000     0.0
205  0.000000  0.000000  0.000000    0.0  0.0   0.0  0.000000     0.0
206  0.000000  0.000000  0.000000    0.0  0.0   0.0  0.023620     0.0
207  0.017845  0.000000  0.000000    0.0  0.0   0.0  0.014268     0.0
writing     wrong       www   xgboost   xi  yeah      year  yearly  \
0        0.0  0.000000  0.000000  0.009264  0.0   0.0  0.000000     0.0
1        0.0  0.000000  0.025664  0.000000  0.0   0.0  0.000000     0.0
2        0.0  0.010865  0.013686  0.000000  0.0   0.0  0.006687     0.0
3        0.0  0.000000  0.000000  0.000000  0.0   0.0  0.000000     0.0
4        0.0  0.000000  0.000000  0.000000  0.0   0.0  0.000000     0.0
..       ...       ...       ...       ...  ...   ...       ...     ...
203      0.0  0.024843  0.000000  0.000000  0.0   0.0  0.020386     0.0
204      0.0  0.040656  0.076815  0.000000  0.0   0.0  0.012511     0.0
205      0.0  0.000000  0.000000  0.000000  0.0   0.0  0.000000     0.0
206      0.0  0.000000  0.014339  0.000000  0.0   0.0  0.014012     0.0
207      0.0  0.010315  0.000000  0.000000  0.0   0.0  0.006348     0.0
yellow       yes  yesterday  yield        yo  york      you     young  \
0       0.0  0.000000        0.0    0.0  0.000000   0.0  0.00000  0.000000
1       0.0  0.000000        0.0    0.0  0.000000   0.0  0.00000  0.000000
2       0.0  0.000000        0.0    0.0  0.000000   0.0  0.00000  0.000000
3       0.0  0.006571        0.0    0.0  0.000000   0.0  0.00000  0.000000
4       0.0  0.000000        0.0    0.0  0.000000   0.0  0.00000  0.000000
..      ...       ...        ...    ...       ...   ...      ...       ...
203     0.0  0.000000        0.0    0.0  0.000000   0.0  0.00000  0.010093
204     0.0  0.021158        0.0    0.0  0.000000   0.0  0.03434  0.000000
205     0.0  0.000000        0.0    0.0  0.000000   0.0  0.00000  0.000000
206     0.0  0.000000        0.0    0.0  0.000000   0.0  0.00000  0.000000
207     0.0  0.021471        0.0    0.0  0.040204   0.0  0.00000  0.000000
youtube      zero  zhou  zip    zombie      zone  zoom  zuckerberg  \
0        0.0  0.000000   0.0  0.0  0.000000  0.000000   0.0         0.0
1        0.0  0.000000   0.0  0.0  0.000000  0.000000   0.0         0.0
2        0.0  0.000000   0.0  0.0  0.000000  0.000000   0.0         0.0
3        0.0  0.006363   0.0  0.0  0.000000  0.000000   0.0         0.0
```

[Output continues below...]

```
4        0.0  0.000000   0.0  0.0  0.000000  0.000000   0.0         0.0
..       ...       ...   ...  ...       ...       ...   ...         ...
203      0.0  0.000000   0.0  0.0  0.000000  0.000000   0.0         0.0
204      0.0  0.000000   0.0  0.0  0.000000  0.068679   0.0         0.0
205      0.0  0.021420   0.0  0.0  0.000000  0.000000   0.0         0.0
206      0.0  0.000000   0.0  0.0  0.021434  0.019230   0.0         0.0
207      0.0  0.000000   0.0  0.0  0.000000  0.000000   0.0         0.0
TITLE   ID
0     Ensemble methods: bagging, boosting and stacking    1
1                        Understanding AUC - ROC Curve    2
2    How to work with object detection datasets in ...    3
3    11 Dimensionality reduction techniques you sho...    4
4                          The Time Series Transformer    5
..                                                 ...  ...
203    Type 2 Diabetes Reversal  The Quick Start Guide  210
204             How a 22 Day Water Fast Changed My Life  211
205                                Breaking Your Fast  212
206           11 Unusual Tips for How to Wake Up Early  213
207  The 3 Biggest Mistakes Women Make On The Ketog...  214
[208 rows x 3709 columns]
```

Code

```
tsne = TSNE(n_components=2)
tsne_tfidf_features =
tsne.fit_transform(tfidf_features_df[tfidf_vectorizer.get_feature_names_out()].values)
tsne_tfidf_features_df = pd.DataFrame(tsne_tfidf_features)
tsne_tfidf_features_df.columns = ["C1", "C2"]
tsne_tfidf_features_df["TITLE"] = tfidf_features_df["TITLE"]
```

Output

```
C1         C2                                              TITLE
0    -9.315989 -15.296190   Ensemble methods: bagging, boosting and stacking
1   -11.833191 -16.125834                      Understanding AUC - ROC Curve
2    -2.469935 -11.880694  How to work with object detection datasets in ...
3    -4.610616 -19.015120  11 Dimensionality reduction techniques you sho...
4     8.913444 -13.528582                        The Time Series Transformer
..         ...        ...                                             ...
203  15.811986   4.239884    Type 2 Diabetes Reversal  The Quick Start Guide
204  12.799903   3.785853            How a 22 Day Water Fast Changed My Life
205  13.766661   4.198017                                  Breaking Your Fast
206   7.564369   3.245051           11 Unusual Tips for How to Wake Up Early
207  15.187894   7.680935  The 3 Biggest Mistakes Women Make On The Ketog...
[208 rows x 3 columns]
```

Code

```
title = "T-distributed Stochastic Neighbor Embedding for TFIDF document representation"
fig = px.scatter(tsne_tfidf_features_df, x="C1", y="C2", hover_data=['TITLE'], title=title)
fig.show()
```

Output

```
if (window.MathJax && window.MathJax.Hub && window.MathJax.Hub.Config)
{window.MathJax.Hub.Config({SVG: {font: "STIX-Web"}});}               window.PlotlyConfig =
{MathJaxConfig: 'local'};
window.PLOTLYENV=window.PLOTLYENV || {};                                    if
(document.getElementById("01c44fa7-01a7-4ef6-b5ce-23dafe459fb5")) {
Plotly.newPlot(                       "01c44fa7-01a7-4ef6-b5ce-23dafe459fb5",
[{"customdata":[["Ensemble methods: bagging, boosting and stacking"],["Understanding AUC - ROC
Curve"],["How to work with object detection datasets in COCO format"],["11 Dimensionality
reduction techniques you should know in 2021"],["The Time Series Transformer"],["Learning a
Personalized Homepage"],["6 Data Science Certificates To Level Up Your Career"],["Transformers
Explained Visually (Part 2): How it works, step-by-step"],["60 Python Projects with Source
Code"],["Geometric foundations of Deep Learning"],["Machine Learning Basics with the K-Nearest
Neighbors Algorithm"],["Building RNN, LSTM, and GRU for time series using
PyTorch"],["Algorithms of the Mind"],["4 Reasons Why Economists Make Great Data Scientists
(And Why No One Tells Them)"],["How To Create A Chatbot with Python & Deep Learning In Less
Than An Hour"],["Machine Learning is Fun Part 5: Language Translation with Deep Learning and
the Magic of Sequences"],["Illustrated Guide to LSTM's and GRU's: A step by step
explanation"],["How to go from a Python newbie to a Google Certified TensorFlow Developer
under two months"],["17 Clustering Algorithms Used In Data Science and Mining"],["Introduction
to Genetic Algorithms  Including Example Code"],["Photoreal Roman Emperor
Project"],["Fundamental Techniques of Feature Engineering for Machine Learning"],["How I Got a
Job at DeepMind as a Research Engineer (without a Machine Learning Degree!)"],["Towards the
end of deep learning and the beginning of AGI"],["Tutorial: Document Classification using
WEKA"],["Understanding Random Forest"],["Probability concepts explained: Maximum likelihood
estimation"],["Transformers Explained Visually (Part 3): Multi-head Attention, deep
dive"],["180 Data Science and Machine Learning Projects with Python"],["5 Online Courses I
Took as a Self-Taught Data Scientist"],["Machine Learning is Fun! Part 4: Modern Face
Recognition with Deep Learning"],["9 Distance Measures in Data Science"],["The Sexiest Job of
the 21st Century Isn't \"Sexy\" Anymore"],["Machine Learning is Fun! Part 3: Deep Learning and
Convolutional Neural Networks"],["A Comprehensive Guide to Convolutional Neural Networks  the
```

ELI5 way"],["20 Necessary Requirements of a Perfect Laptop for Data Science and Machine Learning Tasks"],["3 Beginner Mistakes I've Made in My Data Science Career"],["Understanding Variational Autoencoders (VAEs)"],["Setting Up a New M1 MacBook for Data Science"],["Are The New M1 Macbooks Any Good for Data Science? Let's Find Out"],["Simple and Multiple Linear Regression in Python"],["I tripled my income with data science. Here's how."],["Keyword Extraction process in Python with Natural Language Processing(NLP)"],["Machine learning models for 100% better returns in Algo-trading"],["How to Install Ubuntu Desktop With a Graphical User Interface in WSL2"],["Understanding Contrastive Learning"],["Understanding Semantic Segmentation with UNET"],["How Transformers Work"],["Yes you should understand backprop"],["PCA using Python (scikit-learn)"],["Hyperparameter Tuning the Random Forest in Python"],["Top 3 Reasons Why I Sold My M1 Macbook Pro as a Data Scientist"],["TRAIN A CUSTOM YOLOv4 OBJECT DETECTOR (Using Google Colab)"],["6 Machine Learning Certificates to Pursue in 2021"],["What to do with \"small\" data?"],["Time Series Forecasting with PyCaret Regression Module"],["15 Habits I Learned from Highly Effective Data Scientists"],["OVER 100 Data Scientist Interview Questions and Answers!"],["Activation Functions in Neural Networks"],["A One-Stop Shop for Principal Component Analysis"],["5 Things You Should Know About Covariance"],["How to build your own Neural Network from scratch in Python"],["The best explanation of Convolutional Neural Networks on the Internet!"],["Lambda Functions with Practical Examples in Python"],["The Complete Guide to Time Series Analysis and Forecasting"],["8 Ways to Filter Pandas Dataframes"],["Standing with Dr. Timnit Gebru #ISupportTimnit #BelieveBlackWomen"],["Visualising high-dimensional datasets using PCA and t-SNE in Python"],["How to Study for the Google Data Analytics Professional Certificate"],["Is Data Science Still a Rising Career in 2021"],["Simple Reinforcement Learning with Tensorflow Part 0: Q-Learning with Tables and Neural Networks"],["How To Grow From Non-Coder to Data Scientist in 6 Months"],["Apple's New M1 Chip is a Machine Learning Beast"],["Train\u002fTest Split and Cross Validation in Python"],["Text Classification with NLP: Tf-Idf vs Word2Vec vs BERT"],["The 5 Clustering Algorithms Data Scientists Need to Know"],["Building A Logistic Regression in Python, Step by Step"],["Springer has released 65 Machine Learning and Data books for free"],["Every single Machine Learning course on the internet, ranked by your reviews"],["TensorFlow Tutorial Part 1"],["The 7 Best Data Science and Machine Learning

Podcasts"],["Random Forest in Python"],["How to Land a Data Analytics Job in 6 Months"],["What is a Transformer?"],["Fundamentals Of Statistics For Data Scientists and Analysts"],["Is Google's AI research about to implode?"],["Machine Learning is Fun! Part 2"],["Understanding Generative Adversarial Networks (GANs)"],["17 types of similarity and dissimilarity measures used in data science."],["Deep Learning Is Going to Teach Us All the Lesson of Our Lives: Jobs Are for Machines"],["I interviewed at five top companies in Silicon Valley in five days, and luckily got five job offers"],["Advantages and Disadvantages of Artificial Intelligence"],["Could Nim Replace Python?"],["An End-to-End Project on Time Series Analysis and Forecasting with Python"],["Data Scientists Will be Extinct in 10 Years"],["Import all Python libraries in one line of code"],["Enchanted Random Forest"],["Machine Learning in a Week"],["50+ Statistics Interview Questions and Answers for Data Scientists for 2022"],["Top 10 Data Science Projects for Beginners"],["Check For a Substring in a Pandas DataFrame Column"],["Machine Learning is Fun Part 6: How to do Speech Recognition with Deep Learning"],["Everything You Need to Know About Artificial Neural Networks"],["Ways to Detect and Remove the Outliers"],["How to Crush the Crypto Market, Quit Your Job, Move to Paradise and Do Whatever You Want the Rest of Your Life"],["Bitcoin Is Venice"],["How I Built a Net Worth of $500,000 Before Age 30"],["Nassim Nicholas Taleb on Self-Education and Doing the Math (Ep. 41 Live at Mercatus)"],["Public Mint Polkastarter IDO: Launching 23rd of February, Whitelist Now Open!"],["This is How I Made $40k In Passive Income By Age 26"],["Introducing The Iron Bank"],["I used Acorns, Robinhood, and Stash for 2 years. This is what I learned and earned."],["Explaining blockchain how proof of work enables trustless consensus"],["Wealthfront:Silicon Valley Tech at Wall Street Prices"],["Crypto Trading Bots A helpful guide for beginners [2020]"],["Why People Still Don't Get Cryptocurrency"],["Australia's Economy is a House of Cards"],["My First Two Months Trading Stocks with Robinhood"],["What It Takes to Go from $0 to $1 Million in Less Than One Year"],["Tesla Is Dead (And Elon Musk Knows It)"],["Uber's Credit Card Is Bankrupting Restaurants... and It's All Your Fault"],["A Quick Starter Guide to Leveraged Trading at BitMEX"],["I Made $3 Million in Crypto. These are the 26 Rules I Learned."],["The Black-Scholes formula, explained"],["You Will Never Be Rich If You Keep Doing These 10 things"],["Heroes Give, Superheroes Borrow"],["Could Bitcoin's Bull Market End Next

Month?"],["The Exact Steps I Followed to Make $1,500+ of Passive Income Every Month"],["You May Have A Poor Person's Mindset And Not Know It"],["A Definitive Guide to Why Life Is So Terrible for Most Millennials"],["The Berkshire Hathaway of The Internet"],["How to invest in Bitcoin properly. Blockchain and other cryptocurrencies"],["Machine learning in finance: Why, what & how"],["Minimum Wage Artists"],["Richart + @GoGo "],["How to store Bitcoins and other cryptocurrencies properly."],["I was wrong about Ethereum"],["Deep Learning the Stock Market"],["Financial Fridays: It's Financial Suicide To Own A House"],["Gauge Theory Does Not Fix This"],["High Frequency Trading on the Coinbase Exchange"],["A $1000 Bitcoin Investment Won't Make You Rich"],["How To Legally Own Another Person"],["5 Things NOT to Do in the Robinhood App for Stock Trading"],["The One Word That Explains Why Economics Professors Are Not Billionaires"],["Building your credit history"],["I Retired at 35 Here Are 5 Lessons from My First 6 Months of Freedom"],["Why you should never use Upwork, ever."],["Facebook Can't Be Fixed."],["A comprehensive guide to downloading stock prices in Python"],["Detecting Credit Card Fraud Using Machine Learning"],["2018 & : KOKO COMBO icash "],["Is Wealthfront Worth it?"],["How I Slowly Became A \"Middle-Class\" Millionaire"],["SPY vs. QQQ: Investing in Different Indexes"],["I Won $104 Million for Blowing the Whistle on My CompanyBut Somehow I Was the Only One Who Went to Jail"],["Robinhood Lends \"Your\" Shares to Short Sellers (and Keeps All the Proceeds)"],["The Secret to Making 2000% in Stocks Overnight, the Anavex story."],["4 Unusual Side Hustles I Do to Earn an Extra $1,500 a Month"],["Meet 'Spoofy'. How a Single entity dominates the price of Bitcoin."],["How I Started Saving a TON of Money."],["How I transformed $6,000 to $3,000,000"],["Cliff Asness on Marvel vs. DC and Why Never to Share a Gym with Cirque du Soleil (Ep. 5 Live at Mason)"],["Paypal froze our funds, then offered us a business loan"],["A Visual Explanation of Algorithmic Stablecoins"],["Algorithmic Trading Bot: Python"],["Chernobyl's Blown Up Reactor 4 Just Woke Up"],["The Unexpected Case of the Disappearing Flu"],["The Diet & Workout Plan of a Full-Time Traveling Family HIS"],["The Cult of Work You Never Meant to Join"],["5 Things to do in the First 24 Hours of a Cold or Flu"],["The Sweet Spot for Intermittent Fasting"],["The 6 Types of ICU Nurse"],["How to biohack your intelligence with everything from sex to modafinil to MDMA"],["8 Common Arguments Against Vaccines"],["A Reasonably Detailed Guide to Optimizing Your iPhone for Productivity, Focus and Your Own Health"],["The cure for type 2 diabetes is

known, but few are aware"],["Face mapping: how to deal with acne like a true detective"],["I'm 32 and spent $200k on biohacking. Became calmer, thinner, extroverted, healthier & happier."],["I need more iron"],["What I Learned From Quitting Coffee After 15 Years Of Daily Consumption"],["The forgotten art of untucking the tail"],["Hip Replacement Isn't What It Used To Be 12 Weeks Ago"],["Coronavirus: Why You Must Act Now"],["Manufacturers have been using nanotechnology-derived graphene in face masks now there are safety concerns"],["Eben Byers: The Man Who Drank Radioactive Water Until His Jaw Fell Off"],["My Intermittent Fasting Lifestyle: How I Dropped 50 Pounds"],["I Stopped Drinking for 30 Days. Here's What Happened."],["How to lose 10+ pounds of fat a month- even if you have a slow metabolism"],["If You're Buying Your Weed Solely Based on the THC Level, You're Doing it Wrong."],["2016 Is Not Killing People"],["How I Learned to Sleep Only Three Hours Per Night (and Why You Should Too)"],["I fasted for 11 days, here is what happened."],["Coronavirus: The Hammer and the Dance"],["What Happens in Your Body When You Quit Drinking"],["The Science Behind Fat Metabolism"],["The Easiest Way to Lose 125 Pounds Is to Gain 175 Pounds"],["7 things I did to reboot my life."],["How I lost 10kg in 60 days: My 7-step weight loss plan"],["The Foo Fighters' AIDS denialism should be on the record"],["The Uncut History of Male Circumcision"],["10 Things You Can Do This Morning To Heal Your Anxiety"],["I just lost 100 pounds. Here's whyalmost nobody else will!"],["Type 2 Diabetes Reversal The Quick Start Guide"],["How a 22 Day Water Fast Changed My Life"],["Breaking Your Fast"],["11 Unusual Tips for How to Wake Up Early"],["The 3 Biggest Mistakes Women Make On The Ketogenic Diet (And How To Fix
Them)"]],"hovertemplate":"C1=%{x}\u003cbr\u003eC2=%{y}\u003cbr\u003eTITLE=%{customdata[0]}\u003cextra\u003e\u003c\u002fextra\u003e","legendgroup":"","marker":{"color":"#636efa","symbol":"circle"},"mode":"markers","name":"","orientation":"v","showlegend":false,"x":[-9.315989,-11.833191,-2.4699347,-4.610616,8.913444,-2.9239318,-11.590107,8.049854,-4.11645,4.7490964,-3.1185791,-4.8027296,1.4914281,-9.670849,0.47556853,5.512588,7.175657,-11.736506,-10.0620775,-11.107927,2.3328247,-5.005572,3.9932432,1.4339564,-1.4392763,-11.531571,-9.226301,8.642264,-5.0547066,-8.640053,3.3035727,-1.7443551,-9.0209,3.2825568,2.6761878,-0.7996011,-6.2886343,-0.42953196,0.18807518,-1.3544536,-6.9306145,-7.525899,-2.5576305,7.6714096,-0.8586246,0.79306644,2.3078864,6.593957,7.0923133,-4.3014164,-8.182425,-1.1375114,-1.6360904,-

11.247784,-8.092777,-6.1719527,-7.884475,-8.965762,1.2476798,-6.0238104,-6.6851444,4.9449224,4.1302958,0.74279714,-5.276898,-2.3810246,-5.566096,-3.8122823,-8.113997,-9.474904,5.634501,-8.225846,0.21408634,-6.953915,5.097691,-10.194088,-6.8356733,-13.577475,-9.187543,-12.038179,-11.571358,-8.81773,-6.960813,7.7211885,-8.133144,2.8636918,2.8976247,0.26068002,-1.9724907,3.4096043,4.046223,3.9269056,-3.8021204,-5.2877164,-9.289019,-3.5194232,-11.881662,-9.17181,-9.234589,-6.272526,-2.3119836,2.8465633,5.405137,-5.7272186,0.7836245,-2.5327766,-0.259112,4.622437,-2.1767406,2.0221038,-0.80968654,3.8464122,-8.946898,2.1650531,2.5096295,-0.6155701,-3.508444,4.657402,1.4426056,7.517956,-6.1202893,1.2388966,0.21829088,7.197798,-0.455204,-3.9003506,-1.3760433,1.1520145,-0.39317012,-1.5086403,2.8900056,-3.0973501,-8.097268,-1.1099837,-8.936979,-4.1326294,-1.5063623,6.0429897,-2.8908067,-3.5670478,2.4042907,-1.723819,3.1268907,5.106994,-1.7209702,-8.550798,0.43234056,-4.396475,-5.042144,8.577991,-8.773923,-9.198908,3.1265879,-1.5155245,3.8533468,-4.4690557,5.0134645,5.7752247,-3.1170964,0.19785751,-1.574031,6.052507,3.4655333,-8.481684,-1.0858274,8.52773,13.081094,3.5405545,11.76582,7.492216,4.757832,16.463934,7.2931013,9.0738125,2.5713732,5.209379,15.092896,8.8397455,10.017846,-0.5598702,10.110757,12.241174,8.333039,3.1674306,12.187223,13.073403,12.937368,10.454109,13.152882,7.4957647,0.13759266,7.45174,12.31172,3.145226,11.066696,15.078258,10.091437,8.936834,12.581308,0.5511121,6.6297336,7.8065615,11.509631,15.811986,12.799903,13.766661,7.5643687,15.187894],"xaxis":"x","y":[-15.29619,-16.125834,-11.880694,-19.01512,-13.528582,-25.30444,-4.7107863,-12.177921,-6.042511,-15.930358,-16.334589,-13.644135,-14.906568,-2.2172763,-11.412706,-10.863135,-15.489435,-6.848895,-22.007883,-18.436022,-20.479975,-14.837585,-2.0375588,-8.677378,-13.436925,-12.946169,-20.014368,-12.398086,-6.1198936,-3.297544,-14.685987,-17.282852,-0.49332437,-15.7138,-17.050507,-1.1610122,-0.7121348,-20.184757,-3.0853746,-3.0855758,-18.280632,-1.2679317,-6.900768,14.775976,-10.8327675,-15.16726,-16.753952,-12.302652,-17.683044,-19.417263,-12.958015,-2.3751276,-11.719664,-4.7246475,-14.772839,-12.862541,0.35852203,-17.230522,-23.860062,-19.783014,-20.919716,-18.393635,-17.30112,-23.987207,-10.744829,-24.28197,0.9308304,-19.924427,-4.0153103,-0.72944105,-19.527359,-2.0881457,-2.7903688,-13.261011,-12.548573,-22.249113,-17.270079,-11.734575,-5.248601,-7.535642,-1.7476407,-13.217513,-3.6616926,-11.550816,-18.725414,-10.2409525,-

10.870039,-20.29302,-17.159985,-5.969034,-2.0624053,-6.1652837,-6.361189,-
10.385512,0.56754243,-4.1623826,-12.730082,-6.700504,-17.615473,-3.3560197,-23.76958,-
13.016477,-17.397022,-
15.717107,15.174789,16.936749,8.803946,10.442505,21.30963,12.294472,24.004065,16.20506,-
9.735259,13.487444,20.619522,13.466934,9.971961,16.947392,6.8648305,17.280552,10.444657,17.519
15,17.085987,13.502919,10.703545,12.900253,16.930792,9.062825,12.393475,8.157716,8.417664,17.8
9453,-6.3124895,6.727773,7.448571,18.548143,19.39053,-
13.432589,9.793413,15.693049,20.549309,17.360159,10.04226,16.69422,12.875428,14.409708,10.5360
02,5.194278,13.143711,16.262674,-
10.805488,7.500324,12.733348,9.594695,13.828485,8.489754,18.183538,14.690075,6.4473004,18.7291
6,11.229829,15.68807,11.787925,14.744705,22.986956,14.891694,-
4.1958537,1.6721252,7.761608,5.032597,1.2905202,5.9163165,7.305699,4.1715565,4.775253,5.496660
7,5.7459736,-3.6335888,4.0625243,24.775711,-0.20017174,9.246378,7.4983897,3.1386008,-
4.4892693,1.8194675,4.6138396,0.69618684,6.2792172,9.95257,3.2342596,4.2641635,2.9998052,3.023
7696,1.0165321,7.077837,6.2935677,5.833287,7.0771465,2.8447852,0.55871594,2.4379056,6.3448954,
4.239884,3.785853,4.1980166,3.2450507,7.680935],"yaxis":"y","type":"scatter"}],
{"template":{"data":{"histogram2dcontour":[{"type":"histogram2dcontour","colorbar":{"outlinewi
dth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.22222222222
22222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.555555555555
5556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888
888,"#fdca26"],[1.0,"#f0f921"]]}],"choropleth":[{"type":"choropleth","colorbar":{"outlinewidth
":0,"ticks":""}}],"histogram2d":[{"type":"histogram2d","colorbar":{"outlinewidth":0,"ticks":""
},"colorscale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],
[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[
0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1
.0,"#f0f921"]]}],"heatmap":[{"type":"heatmap","colorbar":{"outlinewidth":0,"ticks":""},"colors
cale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.333333
3333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666
666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f9
21"]]}],"heatmapgl":[{"type":"heatmapgl","colorbar":{"outlinewidth":0,"ticks":""},"colorscale"

:[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]}],"contourcarpet":[{"type":"contourcarpet","colorbar":{"outlinewidth":0,"ticks":""}}],"contour":[{"type":"contour","colorbar":{"outlinewidth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]}],"surface":[{"type":"surface","colorbar":{"outlinewidth":0,"ticks":""},"colorscale":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]]}],"mesh3d":[{"type":"mesh3d","colorbar":{"outlinewidth":0,"ticks":""}}],"scatter":[{"fillpattern":{"fillmode":"overlay","size":10,"solidity":0.2},"type":"scatter"}],"parcoords":[{"type":"parcoords","line":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scatterpolargl":[{"type":"scatterpolargl","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"bar":[{"error_x":{"color":"#2a3f5f"},"error_y":{"color":"#2a3f5f"},"marker":{"line":{"color":"#E5ECF6","width":0.5},"pattern":{"fillmode":"overlay","size":10,"solidity":0.2}},"type":"bar"}],"scattergeo":[{"type":"scattergeo","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scatterpolar":[{"type":"scatterpolar","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"histogram":[{"marker":{"pattern":{"fillmode":"overlay","size":10,"solidity":0.2}},"type":"histogram"}],"scattergl":[{"type":"scattergl","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scatter3d":[{"type":"scatter3d","line":{"colorbar":{"outlinewidth":0,"ticks":""}},"marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scattermapbox":[{"type":"scattermapbox","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scatterternary":[{"type":"scatterternary","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"scattercarpet":[{"type":"scattercarpet","marker":{"colorbar":{"outlinewidth":0,"ticks":""}}}],"carpet":[{"aaxis":{"endlinecolor":"#2a3f5f","gridcolor":"white","linecolor":"white","minorgridcolor":"white","startlinecolor":"#2a3f5f"},"baxis":{"endlinecolor":"#2a3f5f","gridcolor":"white","linecolor":"white","minorgridcolor":"white","startlinecolor":"#2a3f5f"},"type":"carpet"}],"table":[{"cells":{"fill":{"color":"#EBF0F8"},"line":{"color":"white"}},"header":{"fill":{"color":"#C

8D4E3"},"line":{"color":"white"}},"type":"table"}],"barpolar":[{"marker":{"line":{"color":"#E5ECF6","width":0.5},"pattern":{"fillmode":"overlay","size":10,"solidity":0.2}},"type":"barpolar"}],"pie":[{"automargin":true,"type":"pie"}]},"layout":{"autotypenumbers":"strict","colorway":["#636efa","#EF553B","#00cc96","#ab63fa","#FFA15A","#19d3f3","#FF6692","#B6E880","#FF97FF","#FECB52"],"font":{"color":"#2a3f5f"},"hovermode":"closest","hoverlabel":{"align":"left"},"paper_bgcolor":"white","plot_bgcolor":"#E5ECF6","polar":{"bgcolor":"#E5ECF6","angularaxis":{"gridcolor":"white","linecolor":"white","ticks":""},"radialaxis":{"gridcolor":"white","linecolor":"white","ticks":""}},"ternary":{"bgcolor":"#E5ECF6","aaxis":{"gridcolor":"white","linecolor":"white","ticks":""},"baxis":{"gridcolor":"white","linecolor":"white","ticks":""},"caxis":{"gridcolor":"white","linecolor":"white","ticks":""}},"coloraxis":{"colorbar":{"outlinewidth":0,"ticks":""}},"colorscale":{"sequential":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]],"sequentialminus":[[0.0,"#0d0887"],[0.1111111111111111,"#46039f"],[0.2222222222222222,"#7201a8"],[0.3333333333333333,"#9c179e"],[0.4444444444444444,"#bd3786"],[0.5555555555555556,"#d8576b"],[0.6666666666666666,"#ed7953"],[0.7777777777777778,"#fb9f3a"],[0.8888888888888888,"#fdca26"],[1.0,"#f0f921"]],"diverging":[[0,"#8e0152"],[0.1,"#c51b7d"],[0.2,"#de77ae"],[0.3,"#f1b6da"],[0.4,"#fde0ef"],[0.5,"#f7f7f7"],[0.6,"#e6f5d0"],[0.7,"#b8e186"],[0.8,"#7fbc41"],[0.9,"#4d9221"],[1,"#276419"]]},"xaxis":{"gridcolor":"white","linecolor":"white","ticks":"","title":{"standoff":15},"zerolinecolor":"white","automargin":true,"zerolinewidth":2},"yaxis":{"gridcolor":"white","linecolor":"white","ticks":"","title":{"standoff":15},"zerolinecolor":"white","automargin":true,"zerolinewidth":2},"scene":{"xaxis":{"backgroundcolor":"#E5ECF6","gridcolor":"white","linecolor":"white","showbackground":true,"ticks":"","zerolinecolor":"white","gridwidth":2},"yaxis":{"backgroundcolor":"#E5ECF6","gridcolor":"white","linecolor":"white","showbackground":true,"ticks":"","zerolinecolor":"white","gridwidth":2},"zaxis":{"backgroundcolor":"#E5ECF6","gridcolor":"white","linecolor":"white","showbackground":true,"ticks":"","zerolinecolor":"white","gridwidth":2}},"shapedefaults":{"line":{"color":"#2a3f5f"}},"annotationdefaults":{"arrowcolor":"#2a3f5f","arrowhead":0,"arrowwidth":1},"geo":{"bgcolor":"white","landcolor":"#E5ECF6","subunitcolor":"white","showland":true,"showlakes":true,"lakecolor":"white"},"title":{"x":0.05},"mapbox":{"style":"light"}}},"xaxis":{"anchor":"y","domain":[0.0,1.0]

```
,"title":{"text":"C1"}},"yaxis":{"anchor":"x","domain":[0.0,1.0],"title":{"text":"C2"}},"legen
d":{"tracegroupgap":0},"title":{"text":"T-distributed Stochastic Neighbor Embedding for TFIDF
document representation"}},                        {"responsive": true}
).then(function(){
var gd = document.getElementById('01c44fa7-01a7-4ef6-b5ce-23dafe459fb5');
var x = new MutationObserver(function (mutations, observer) {{
var display = window.getComputedStyle(gd).display;
if (!display || display === 'none') {{
console.log([gd, 'removed!']);
Plotly.purge(gd);
observer.disconnect();
}}
}});
// Listen for the removal of the full notebook cells
var notebookContainer = gd.closest('#notebook-container');
if (notebookContainer) {{
x.observe(notebookContainer, {childList: true});
}}
// Listen for the clearing of the current output cell
var outputEl = gd.closest('.output');
if (outputEl) {{
x.observe(outputEl, {childList: true});
}}
})               };
```

* In the above scatter plot, we see points usually allong three different directions.

* Hovering over the points, it seems the three directions contains articles broadly on the following topics -

> 1. Data Science/Machine Learning

> 2. Health

> 3. Finance

Code

```
# Let us check top 5 similar articles for some of the articles in our corpus
get_similar_documents(tfidf_features_df, 90, tfidf_vectorizer.get_feature_names_out())
get_similar_documents(tfidf_features_df, 80, tfidf_vectorizer.get_feature_names_out())
get_similar_documents(tfidf_features_df, 150, tfidf_vectorizer.get_feature_names_out())
get_similar_documents(tfidf_features_df, 205, tfidf_vectorizer.get_feature_names_out())
```

Output

```
Reference Article : 17 types of similarity and dissimilarity measures used in data science.
**** Similar Articles ****
9 Distance Measures in Data Science
17 Clustering Algorithms Used In Data Science and Mining
Machine Learning Basics with the K-Nearest Neighbors Algorithm
OVER 100 Data Scientist Interview Questions and Answers!
Fundamental Techniques of Feature Engineering for Machine Learning
Reference Article : TensorFlow Tutorial Part 1
**** Similar Articles ****
How to go from a Python newbie to a Google Certified TensorFlow Developer under two months
Enchanted Random Forest
Building RNN, LSTM, and GRU for time series using PyTorch
PCA using Python (scikit-learn)
Yes you should understand backprop
Reference Article : The One Word That Explains Why Economics Professors Are Not Billionaires
**** Similar Articles ****
You Will Never Be Rich If You Keep Doing These 10 things
Why People Still Don't Get Cryptocurrency
You May Have A Poor Person's Mindset And Not Know It
How I transformed $6,000 to $3,000,000
4 Reasons Why Economists Make Great Data Scientists (And Why No One Tells Them)
Reference Article : How I lost 10kg in 60 days: My 7-step weight loss plan
**** Similar Articles ****
How to lose 10+ pounds of fat a month- even if you have a slow metabolism
10 Things You Can Do This Morning To Heal Your Anxiety
The Diet & Workout Plan of a Full-Time Traveling Family  HIS
The Science Behind Fat Metabolism
The cure for type 2 diabetes is known, but few are aware
```

TFIDF also seems to work well. Notice one improvement in the articles sugested for The One Word That Explains Why Economics Professors Are Not Billionaires
- All 5 articles suggested using TFIDF seems more relevant as conpared to articles suggested by BOW.

* Also notice the improvement in recommendations suggested for TensorFlow Tutorial Part 1

Concluding Remarks

How do we evaluate performance in unsupervised learning problems ?

Validations from - surveys, user feedbacks etc.

How the topics we talked about forms the building blocks of NLP ?

The topics we discussed in this lecture forms the basic building blocks in Natural Language Processing.
- Here we saw that the Medium articles in our corpus mainly belonged to Data Science, Health and Finance.
- In other words there are three (or maybe more) broad topics, that our corpus talks about - this can be seggregated or identified using Topic Modeling.
- The vectorization techniques that we talked about are used extensively in text classification problems as well.
- Further there are more advanced vectorization techniques - which use language models and other deep learning methods - and these are the ones that are State of The Art in most of NLP problems.